MODULE-3
SEARCH IN COMPLEX ENVIRONMENTS

TOPICS

- Adversarial search - Games, Optimal decisions in games, The Minimax algorithm, Alpha-Beta pruning.
- Constraint Satisfaction Problems – Defining CSP, Example Problems
- Constraint Propagation- inference in CSPs
- Backtracking search for CSPs, Structure of CSP problems.

ADVERSARIAL SEARCH

- *In which we examine the problems that arise when we try to plan ahead in a world where other agents are planning against us.*
- *Competitive environments, in which the agents' goals are in conflict*, giving rise to ***adversarial search*** problems—often known as ***games***.
- Examples – Chess, Tic-tac-toe

GAME SEARCH PROBLEM

- Problem formulation
  - Initial state: initial board position + whose move it is
  - Operators: legal moves a player can make
  - Goal (terminal test): game over?
  - Utility (payoff) function: measures the outcome of the game and its desirability
- Search objective:
  - Find the sequence of player's decisions (moves) maximizing its utility (payoff)
  - Consider the opponent's moves and their utility

GAME V/S SINGLE-AGENT SEARCH

- We don’t know how the opponent will act
  - The solution is not a fixed sequence of actions from start state to goal state, but a strategy or policy (a mapping from a state to best move in that state)
- Efficiency is critical to playing well
  - The time to make a move is limited
  - The branching factor, search depth, and number of terminal configurations are huge
    - In chess, branching factor ≈ 35 and depth ≈ 100, giving a search tree of nodes
      - Number of atoms in the observable universe ≈ 1080
    - This rules out searching all the way to the end of the game

- Tools to Handle the Complexity
- ***Pruning*** allows us to ignore portions of the search tree that make no difference to the final choice
- ***Heuristic evaluation functions*** allow us to approximate the true utility of a state without doing a complete search.

- **How AI Plays Games**
  - **Formulate** game as adversarial search problem.
  - Use **minimax search** (maximize my utility, minimize opponent's).
  - Use **pruning** to reduce unnecessary computation.
  - Use **heuristic evaluation** to handle deep trees.
  - The AI doesn't find a fixed path; it maintains a **policy** (best move in every possible position).

MIN MAX PROBLEM

- Consider games with two players, max and min
- MAX moves first, and then they take turns moving until the game is over.
- At the end of the game, points are awarded to the winning player and penalties are given to the loser.
- A game can be formally defined as a kind of search problem with the following elements:

MIN MAX PROBLEM – FORMAL DEFINITION

- **S_0:** The **initial state**, which specifies how the game is set up at the start.
- **Player(s):** defines which player has the move in a state.
- **Actions(s):** Returns the set of legal moves in a state.
- **Result(s, a):** The **transition model**, which defines the result of a move.
- **Terminal-test(s):** A terminal test, which is true when the game is over and false otherwise. States where the games have ended are called **terminal states**
- **Utility(s, p):** A utility function (also called an objective function or payoff function), defines the final numeric value for a game that ends in terminal state s for a player p.(In chess, the outcome is win, lose or a draw with values +1,0,or 1/2.)

GAME TREE

- The initial state, ACTIONS function, and RESULT function define the **game tree** for the game—a tree where the nodes are game states and the edges are moves.
- From the initial state, MAX has nine possible moves.
- Play alternates between ***max's placing an X*** and ***min's placing an O*** until we reach leaf nodes corresponding to terminal states such that **one player has three in a row or all the squares are filled.**
- The number on each leaf node indicates the ***utility value*** of the terminal state from the point of view of max; high values are assumed to be good for max and bad for min (which is how the players get their names)

GAME TREE FOR TIC-TAC-TOE

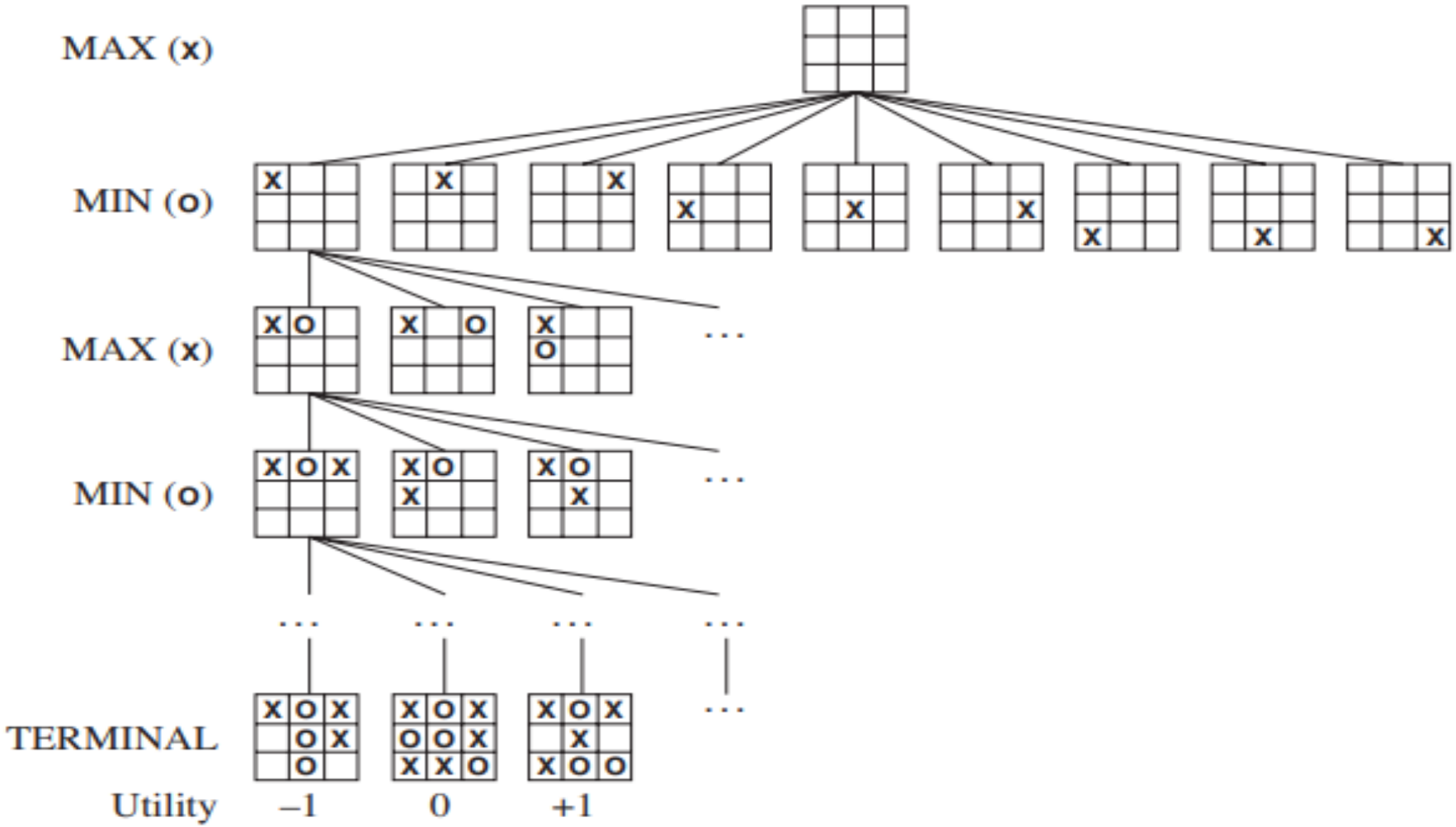


Figure 5.1 A (partial) game tree for the game of tic-tac-toe. The top node is the initial state, and MAX moves first, placing an X in an empty square. We show part of the tree, giving alternating moves by MIN (O) and MAX (X), until we eventually reach terminal states, which can be assigned utilities according to the rules of the game.

- ***Search tree*** *is a tree that is superimposed on the full game tree, and examines enough nodes to allow a player to determine what move to make.*

OPTIMAL SOLUTIONS IN GAMES

- In a normal search problem
  - The optimal solution would be a sequence of actions leading to a goal state—a terminal state that is a win
- In adversarial search
  - 'MIN' interferes the sequence of actions
  - Strategy for 'MAX'
    - Specify moves in the initial state,
    - Observe every possible response by MIN
    - Specify moves in response
    - And so on..
- This optimal strategy can be determined from the **minimax value** of each node

HOW TO CALCULATE THE MINIMAX VALUE?

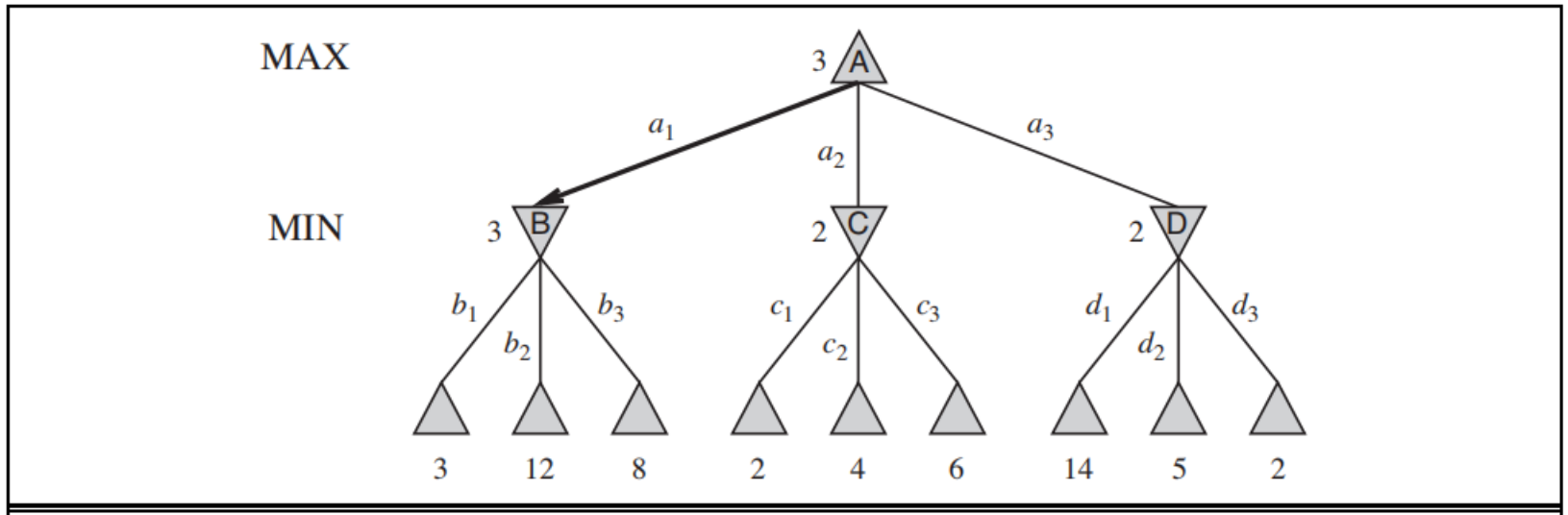


Figure 5.2 A two-ply game tree. The △ nodes are "MAX nodes," in which it is MAX's turn to move, and the ▽ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX's best move at the root is a_1, because it leads to the state with the highest minimax value, and MIN's best reply is b_1, because it leads to the state with the lowest minimax value.

- The possible moves for MAX at the root node are labelled a1, a2, and a3. The possible replies to a1 for MIN are b1, b2, b3, and so on. This particular game ends after one move each by MAX and MIN.
- It is called the two-ply game.(consisting of two half moves)

MINIMAX VALUE

- Given a game tree, the optimal strategy can be determined from the minimax value of each node, which we write as ***minimax(n).***
- The minimax value of a node is of being useful in the corresponding state, *assuming that both players play optimally* from there to the end of the game.
- The minimax value of a terminal state is just its utility(the state of being useful).

Max prefers to move to a state of maximum value, whereas min prefers a state of minimum value.

- The terminal nodes on the bottom level get their utility values from the game's utility function
- **Minimax decision:** optimal choice for max at root that leads to the state with the highest minimax value

MINIMAX VALUE

$$\textsc{Minimax}(s) = \begin{cases} \textsc{Utility}(s) & \text{if } \textsc{Terminal-Test}(s) \\ \max_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{max} \\ \min_{a \in Actions(s)} \textsc{Minimax}(\textsc{Result}(s,a)) & \text{if } \textsc{Player}(s) = \textsc{min} \end{cases}$$

- In the **min max algorithm in AI,** there are two players, Maximiser and Minimiser.

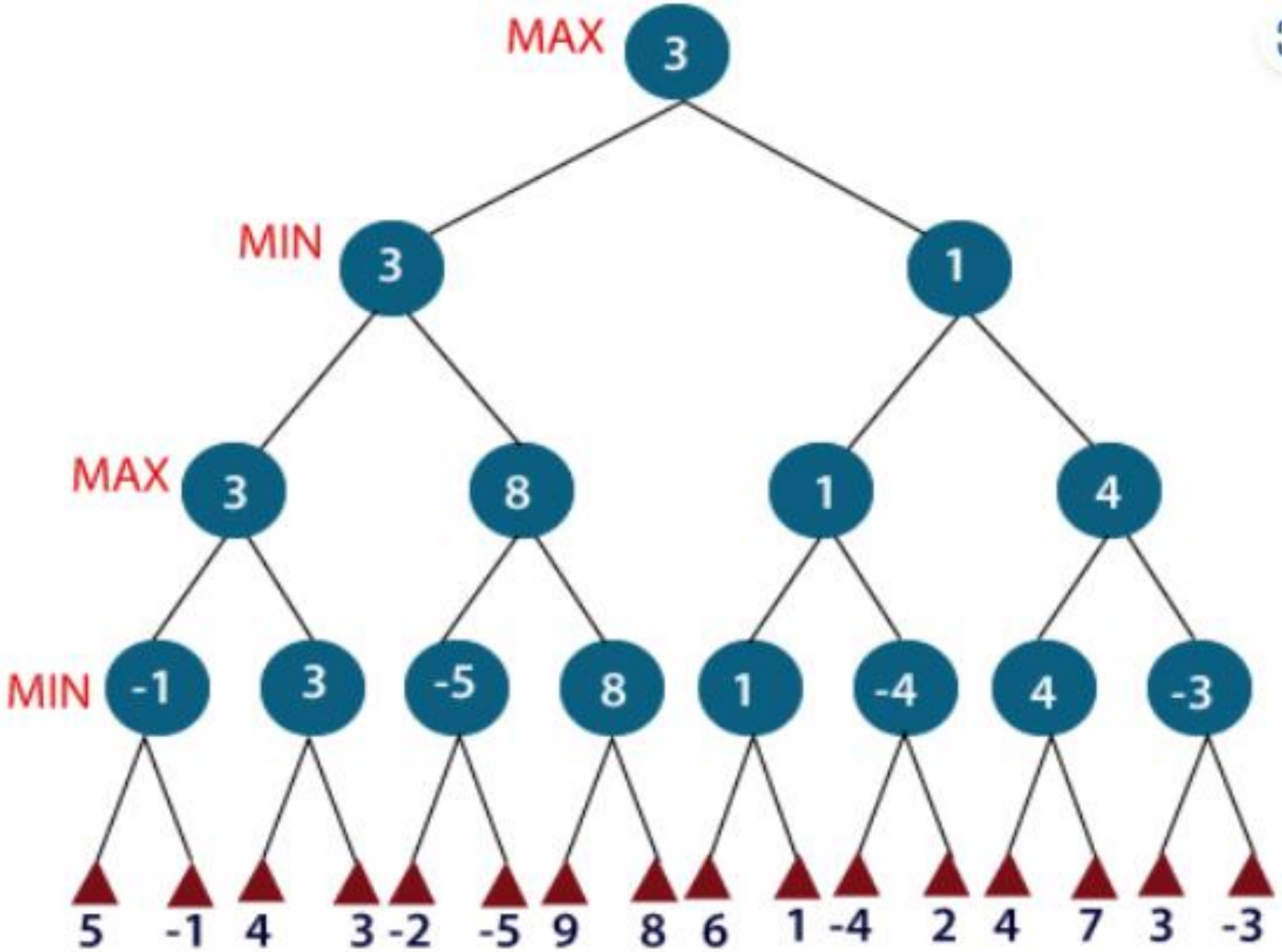
- Both these players play the game as one tries to get the highest score possible or the maximum benefit while the opponent tries to get the lowest score or the minimum benefit.

- Every game board has an evaluation score assigned to it, so the Maximiser will select the maximised value, and the Minimiser will select the minimised value with counter moves.
- If the Maximiser has the upper hand, then the board score will be a positive value, and if the Minimiser has the upper hand, then the board score will be a negative value.

- This is based on the zero-sum game concept where the total utility score gets divided between the two players.

- Thus, an increase in one player's score leads to a decrease in the opponent player's score, making the total score always zero. So, for one player to win, the other has to lose.

- The minimax algorithm performs a ***depth-first search*** algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.
  - Keep on generating the game tree/ search tree till a limit **d**.
  - Compute the move using a heuristic function.
  - Propagate the values from the leaf node till the current position following the minimax strategy.
  - Make the best move from the choices.


MAX
3
MIN
3
1
MAX
3
8
1
4
MIN
-1
3
-5
8
1
-4
4
-3
5
-1
4
3
-2
-5
9
8
6
1
-4
2
4
7
3
-3

- For example, in the above figure, the two players **MAX** and **MIN** are there.
- **MAX** starts the game by choosing one path and propagating all the nodes of that path.
- Now, **MAX** will backtrack to the initial node and choose the best path where his utility value will be the maximum.
- After this, its **MIN** chance. **MIN** will also propagate through a path and again will backtrack, but **MIN** will choose the path which could minimize **MAX** winning chances or the utility value.

- The time complexity of the minimax algorithm is $O(b^m)$.
- The space complexity is O(bm) for an algorithm that generates all actions at once, or O(m) for an algorithm that generates actions one at a time

Minimax algorithm

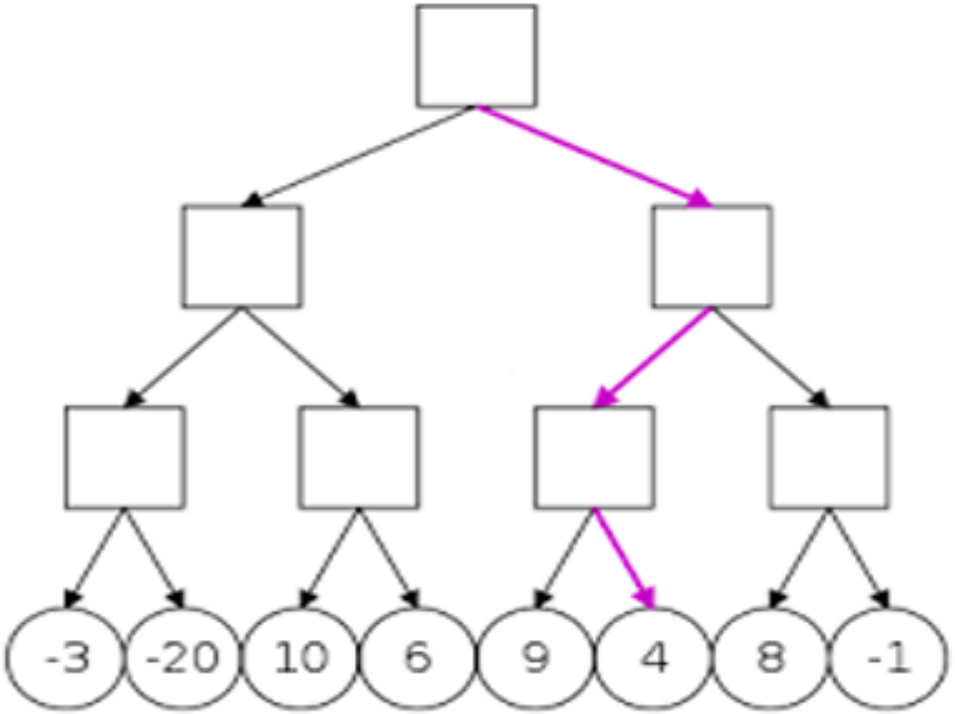
```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\text{argmax}_{a \in S} f(a)$ computes the element a of set S that has the maximum value of $f(a)$.

Apply Minimax search technique to the tree given below, starting with Max player



ALPHA –BETA PRUNING

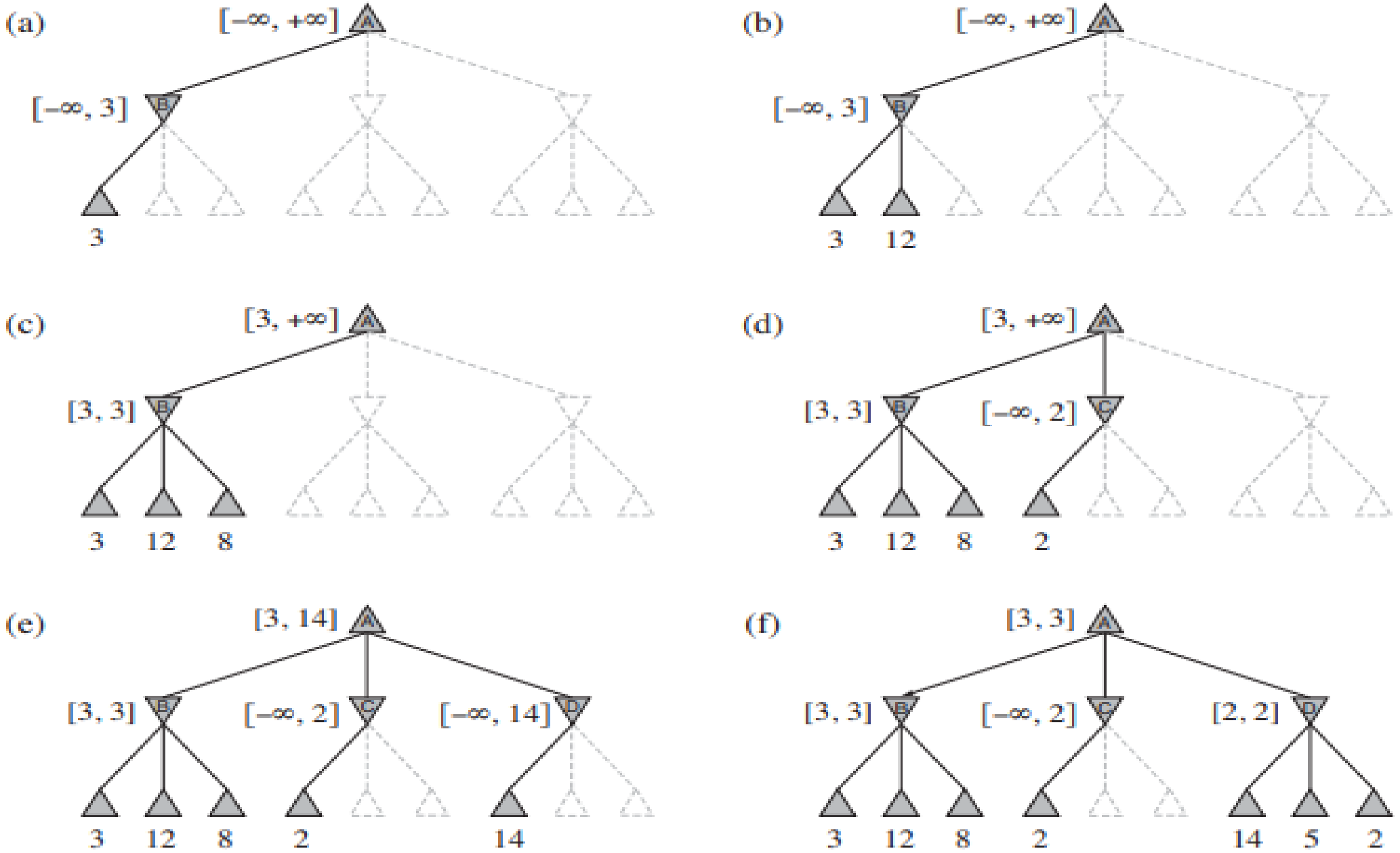
- The problem with the minmax search is that the number of game states it has to examine is exponential in the depth of the tree.
- We can eliminate or do pruning in the game tree
- This particular technique is called ***alpha-beta pruning***
- It can be applied to trees of any depth, possible to prune entire subtrees rather than just leaves
- Alpha- denotes the max value and beta –denotes the min value

Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

$\alpha >= \beta$

- **KEY POINTS**
- **Max** player will only update the value of **alpha.**
- **Min** player will only update the value of **beta.**
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- Only pass the alpha, beta values to the child nodes.


(a)
[−∞, +∞]
A
[−∞, 3]
B
3
(b)
[−∞, +∞]
A
[−∞, 3]
B
3
12
(c)
[3, +∞]
A
[3, 3]
B
3
12
8
(d)
[3, +∞]
A
[3, 3]
B
[−∞, 2]
C
3
12
8
2
(e)
[3, 14]
A
[3, 3]
B
[−∞, 2]
C
[−∞, 14]
D
3
12
8
2
14
(f)
[3, 3]
A
[3, 3]
B
[−∞, 2]
C
[2, 2]
D
3
12
8
2
14
5
2

- Let the two unevaluated successors of node C have values x and y.
- Then the value of the root node is given by

$$\begin{aligned}
\textsc{Minimax}(root) &= \max(\min(3,12,8), \min(2,x,y), \min(14,5,2)) \\
&= \max(3, \min(2,x,y), 2) \\
&= \max(3,z,2) \qquad \text{where } z = \min(2,x,y) \leq 2 \\
&= 3.
\end{aligned}$$

- the value of the root and hence the minimax decision are independent of the values of the pruned leaves x and y

- Alpha–beta pruning gets its name from the following two parameters that describe bounds on the backed-up values that appear anywhere along the path

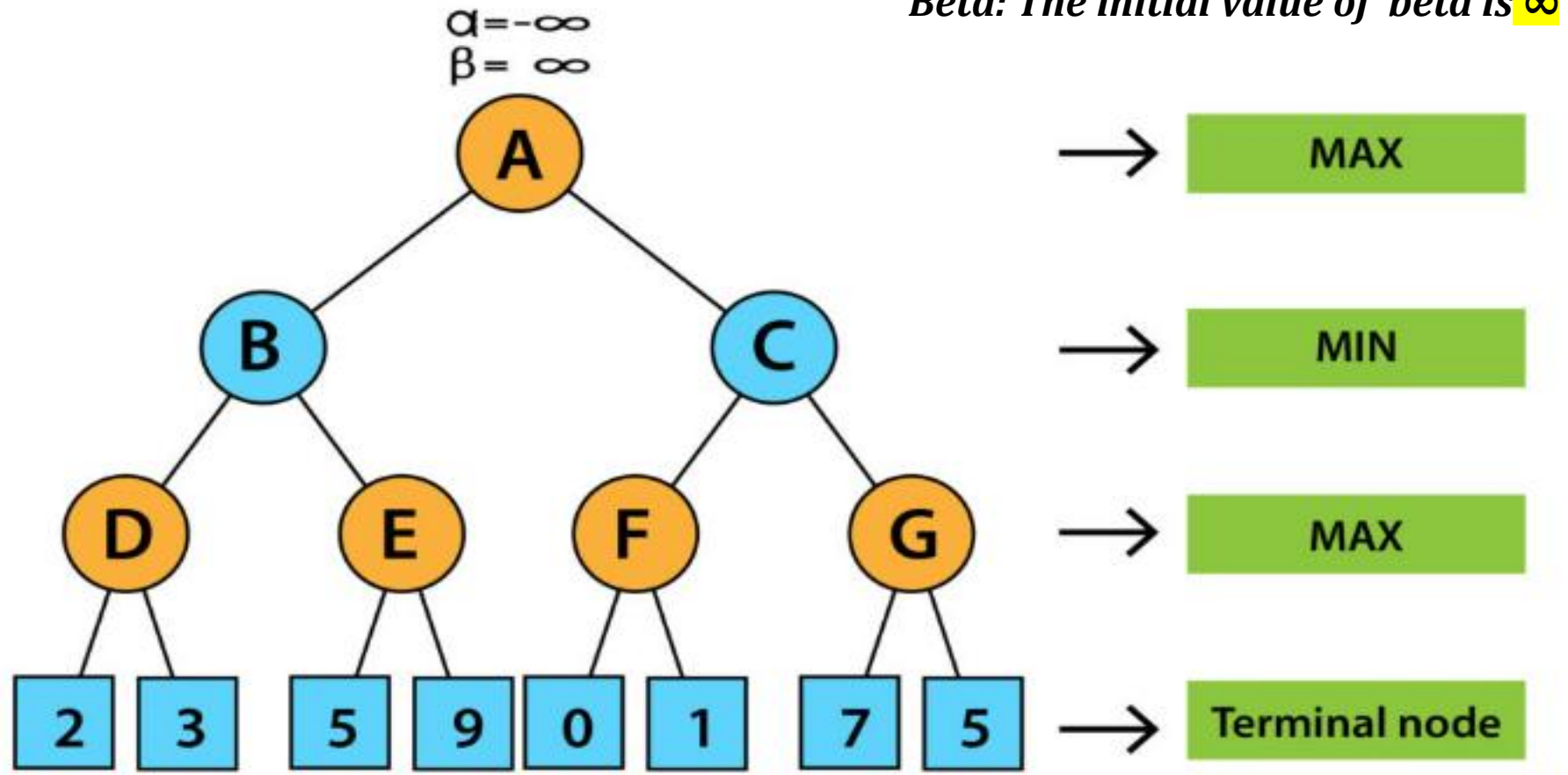
$\alpha =$ the value of the best (i.e., highest-value) choice we have found so far at any choice point along the path for MAX.

$\beta =$ the value of the best (i.e., lowest-value) choice we have found so far at any choice point along the path for MIN.

- Alpha–beta search updates the values of α and β as it goes along and prunes the remaining branches at a node (i.e., terminates the recursive call) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.

EXAMPLE

Alpha: The initial value of alpha is -∞
Beta: The initial value of beta is ∞



Step 1

- At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$, these value of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.

Step 2

- At Node D, the value of α will be calculated as its turn for Max.
- The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.

Step 3

- Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now $\beta = +\infty$, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3 hence at node B now $\alpha = -\infty$, and $\beta = 3$.

- In the next step, algorithm traverse the next successor of Node B which is node E, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.

Step 4

- At node E, Max will take its turn, and the value of alpha will change.

- The current value of alpha will be compared with 5, so max $(-\infty, 5) = 5$, hence at node E $\alpha = 5$ and $\beta = 3$, where $\alpha >= \beta$, so the right successor of E will be pruned, and algorithm will not traverse and the value at node E will be 5.

Step 5

- At next step, algorithm again backtrack the tree, from node B to node A.
- At node A, the value of alpha will be changed the maximum available value is 3 as max $(-\infty, 3) = 3$, and $\beta = +\infty$, these two values now passes to right successor of A which is Node C.
- At node C, $\alpha = 3$ and $\beta = +\infty$, and the same values will be passed on to node F.

Step 6

- At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.

Step 7

- Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed,

- It will compare with 1 so min (∞, 1) = 1.

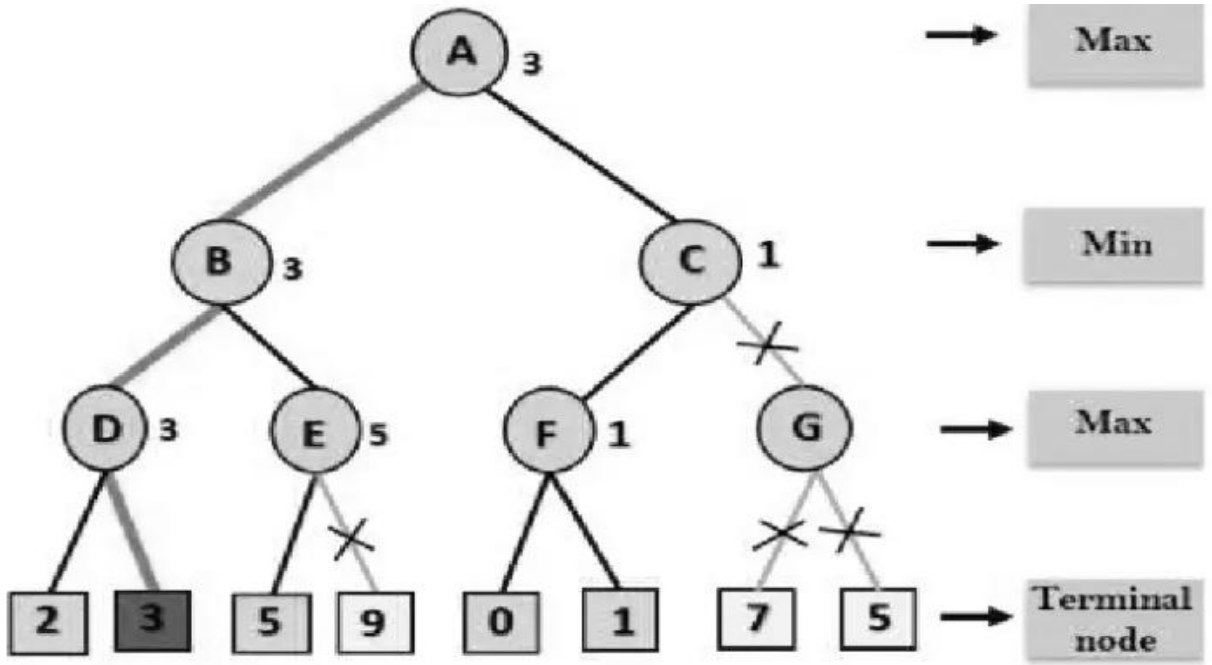
- Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.

Step 8

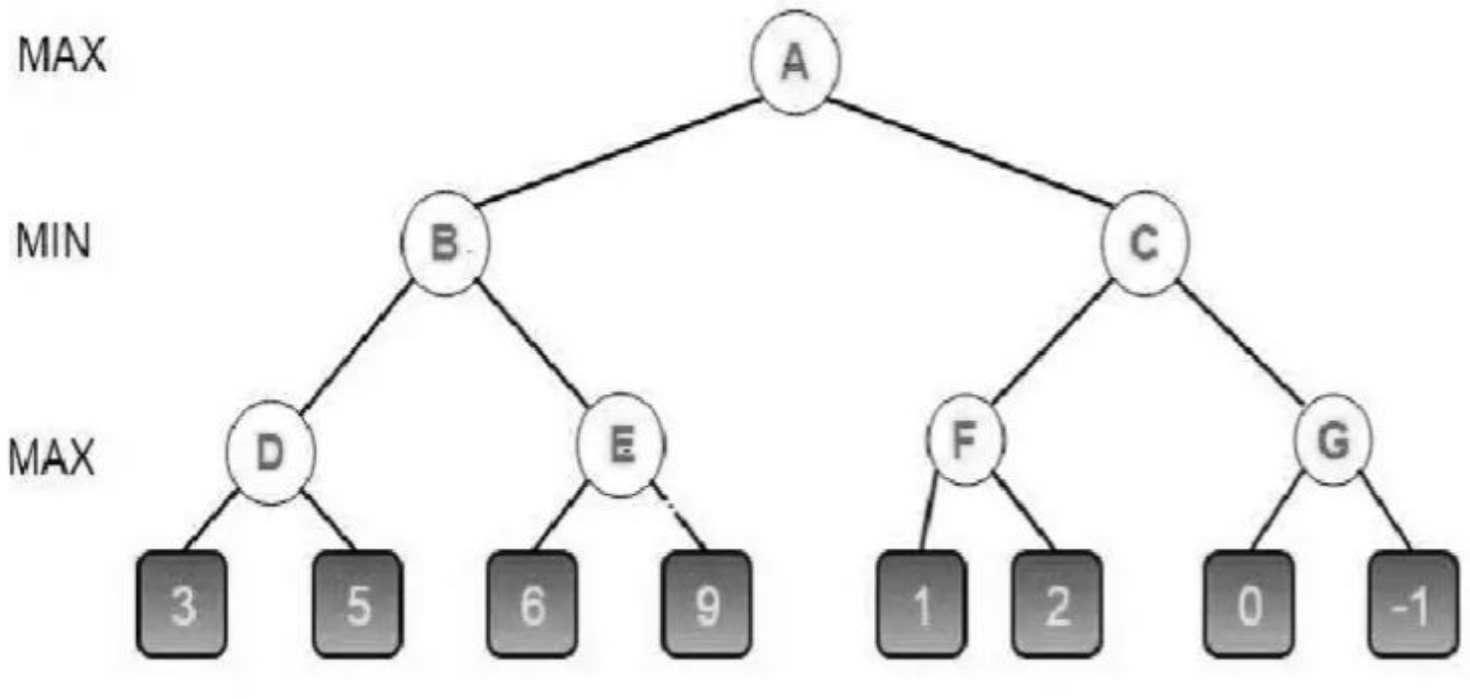
- C now returns the value of 1 to A here the best value for A is max (3, 1) = 3.

- Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed.

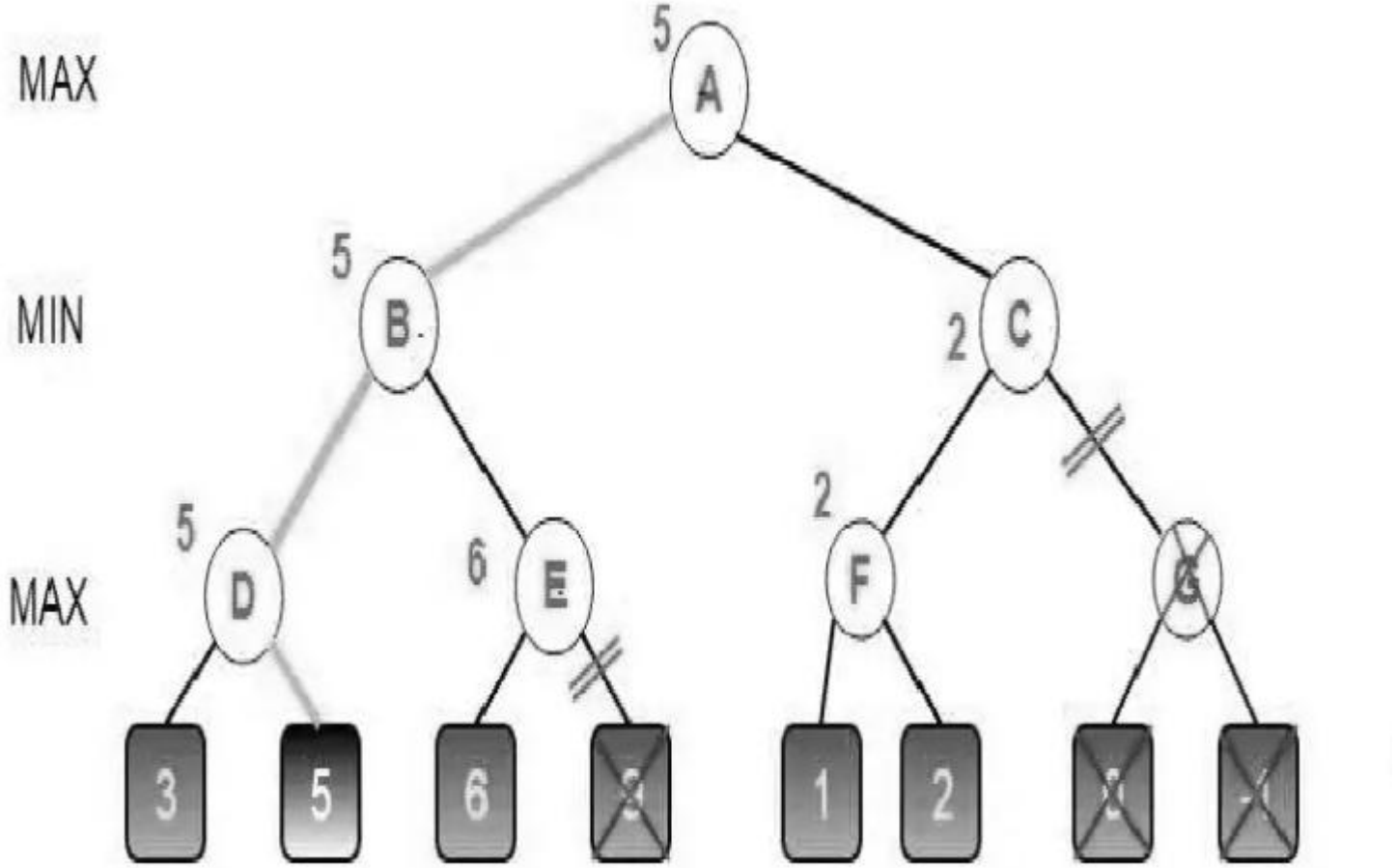
- Hence the optimal value for the maximizer is 3 for this example.



PROBLEM



SOLUTION



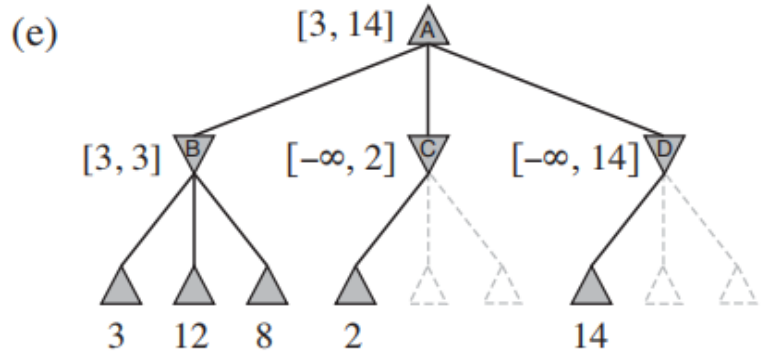
```
function ALPHA-BETA-SEARCH(state) returns an action
    v ← MAX-VALUE(state, −∞, +∞)
    return the action in ACTIONS(state) with value v
```

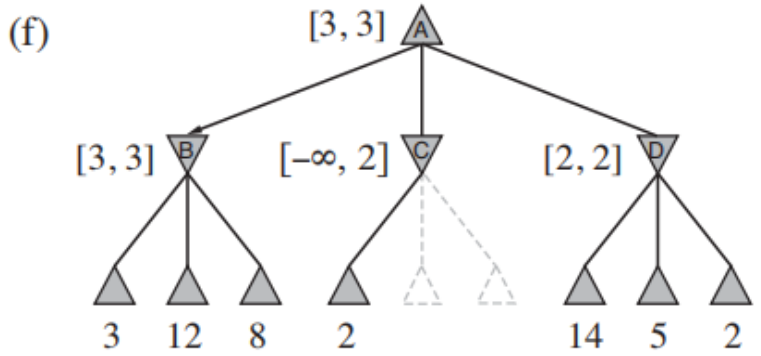
```
function MAX-VALUE(state, α, β) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for each a in ACTIONS(state) do
        v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v
```

```
function MIN-VALUE(state, α, β) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← +∞
    for each a in ACTIONS(state) do
        v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
        if v ≤ α then return v
        β ← MIN(β, v)
    return v
```

MOVE ORDERING

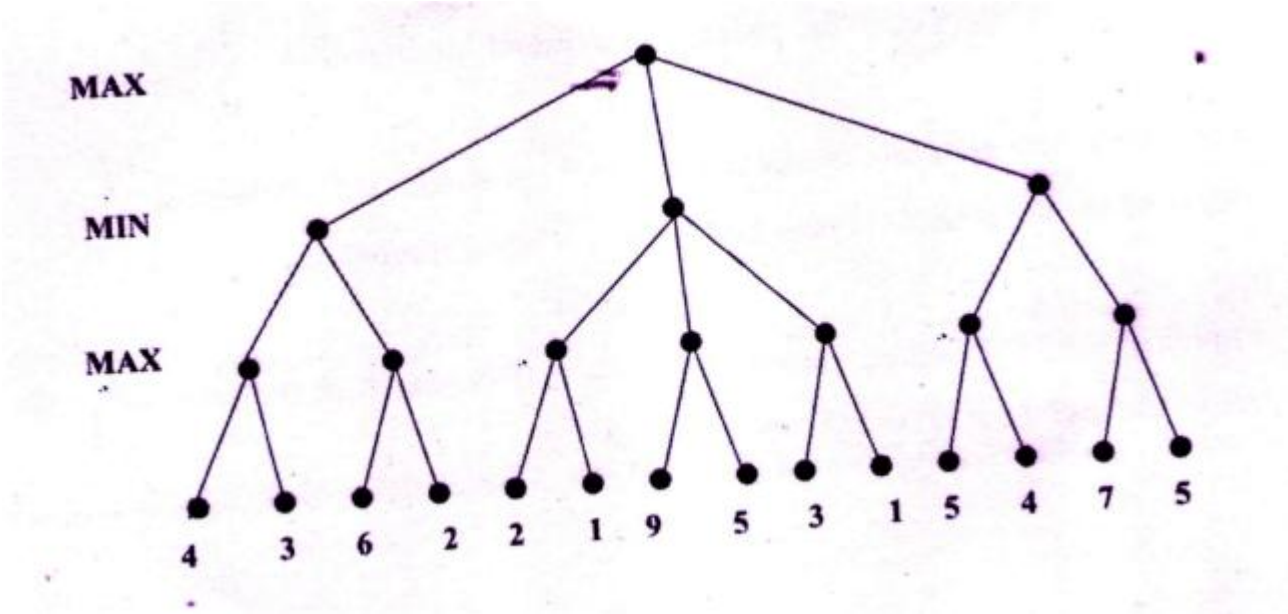
- The effectiveness of alpha–beta pruning is highly dependent on the order in which the states are examined
- Eg, here we could not prune any successors of D at all because the worst successors (from the point of view of MIN) were generated first
- If the third successor of D had been generated first, we would have been able to prune the other two.
- It might be worthwhile to try to examine first the successors that are likely to be best
- The best moves are often called killer moves and to try them first is called the killer move heuristic.





- In many games, repeated states occur frequently because of ***transpositions***
- The hash table of previously seen positions is traditionally called a transposition table; it is essentially identical to the explored list in GRAPH-SEARCH

Perform alpha beta pruning on the given tree



CONSTRAINT SATISFACTION PROBLEMS(CSP)

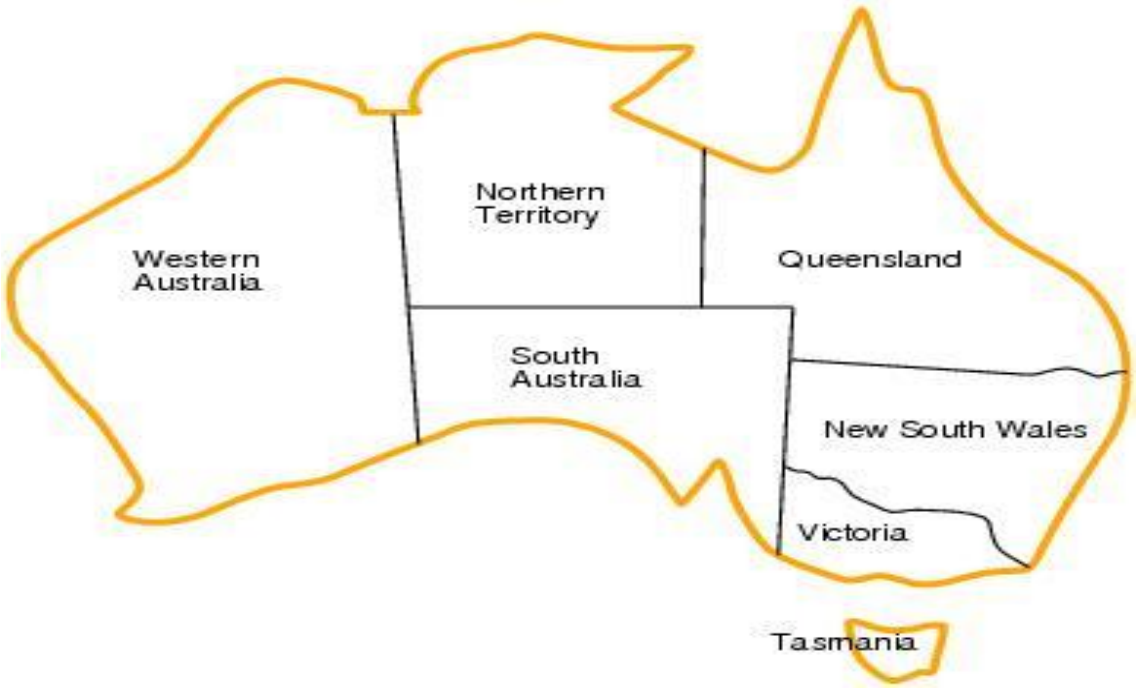
- **A Constraint Satisfaction Problem** is a mathematical problem where the solution must meet a number of constraints.
- In CSP the objective is to ***assign values to variables such that all the constraints are satisfied.***
- Many AI applications use CSPs to solve decision-making problems that involve managing or arranging resources under strict guidelines.
- Common applications of CSPs include:
  - Scheduling: It assigns resources like employees or equipment while respecting time and availability constraints.
  - Planning: Organize tasks with specific deadlines or sequences.
  - Resource Allocation: Distributing resources efficiently without overuse

- CSPs can be used to solve problems such as
  - **Graph-coloring**: given a graph, the a coloring of the graph means assigning each of its vertices a color such that no pair of vertices connected by an edge have the same color
    - In general, this is a very hard problem, e.g. Determining if a graph can be colored with 3 colors is np-hard
    - Many problems boil down to graph coloring, or related problems
  - **Job shop scheduling**: e.g. Suppose you need to complete a set of tasks, each of which have a duration, and constraints upon when they start and stop (e.g. Task c can't start until both task a and task b are finished)
    - CSPs are a natural way to express such problems
  - **Cryptarithmetic puzzles**: e.g. Suppose you are told that TWO + TWO = FOUR, and each of the letters corresponds to a *different* digit from 0 to 9, and that a number can't start with 0 (so T and F are not 0); what, if any, are the possible values for the letters?

- A **constraint satisfaction problem (CSP)** is a problem that requires its solution within some limitations or conditions also known as constraints.
- It consists of the following:
- **A constraint satisfaction problem consists of three components, X,D, and C:**
  - **X is a set of variables, {X1,...,Xn}.**
  - **D is a set of domains, {D1,...,Dn}, one for each variable.**
  - **C is a set of constraints that specify allowable combinations of values.**
  - All these sets should be finite except for the domain set.
  - Each variable in the variable set can have different domains.
  - For example, consider the Sudoku problem again. Suppose that a row, column and block already have 3, 5 and 7 filled in. Then the domain for all the variables in that row, column and block will be {1, 2, 4, 6, 8, 9}.

- **To solve a CSP, we need to define a state space and the notion of a solution.**
- Each state in a CSP is defined by an ***assignment*** of values to some or all of the variables, {Xi = vi , Xj = vj ,...}.
- An assignment that does not violate any constraints is called a ***consistent*** or ***legal assignment.***
- A ***complete assignment*** is one in which every variable is assigned, and a ***solution*** to a CSP is a consistent, complete assignment.
- A ***partial assignment*** is one that assigns values to only some of the variables

EXAMPLE PROBLEM : MAP COLORING



- Given a map of Australia, color it using three colors such that no neighboring territories have the same color.

- To formulate this as a CSP,
- The variables to be the regions X = {WA, NT, Q, NSW ,V, SA, T} .
- The domain of each variable is the set Di = {red, green, blue}.
- The constraints require neighboring regions to have distinct colors.
- Since there are nine places where regions border, there are nine constraints:
- C = {SA ≠ WA, SA ≠ NT, SA ≠ Q, SA ≠ NSW , SA ≠ V, WA ≠ NT, NT ≠ Q, Q ≠ NSW , NSW ≠ V } .

- ***There are many possible solutions to this problem, such as {WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = red }.***

- It can be helpful to visualize a CSP as a ***constraint graph***
- The nodes of the graph correspond to variables of the problem,
- A link connects any two variables that participate in a constraint.

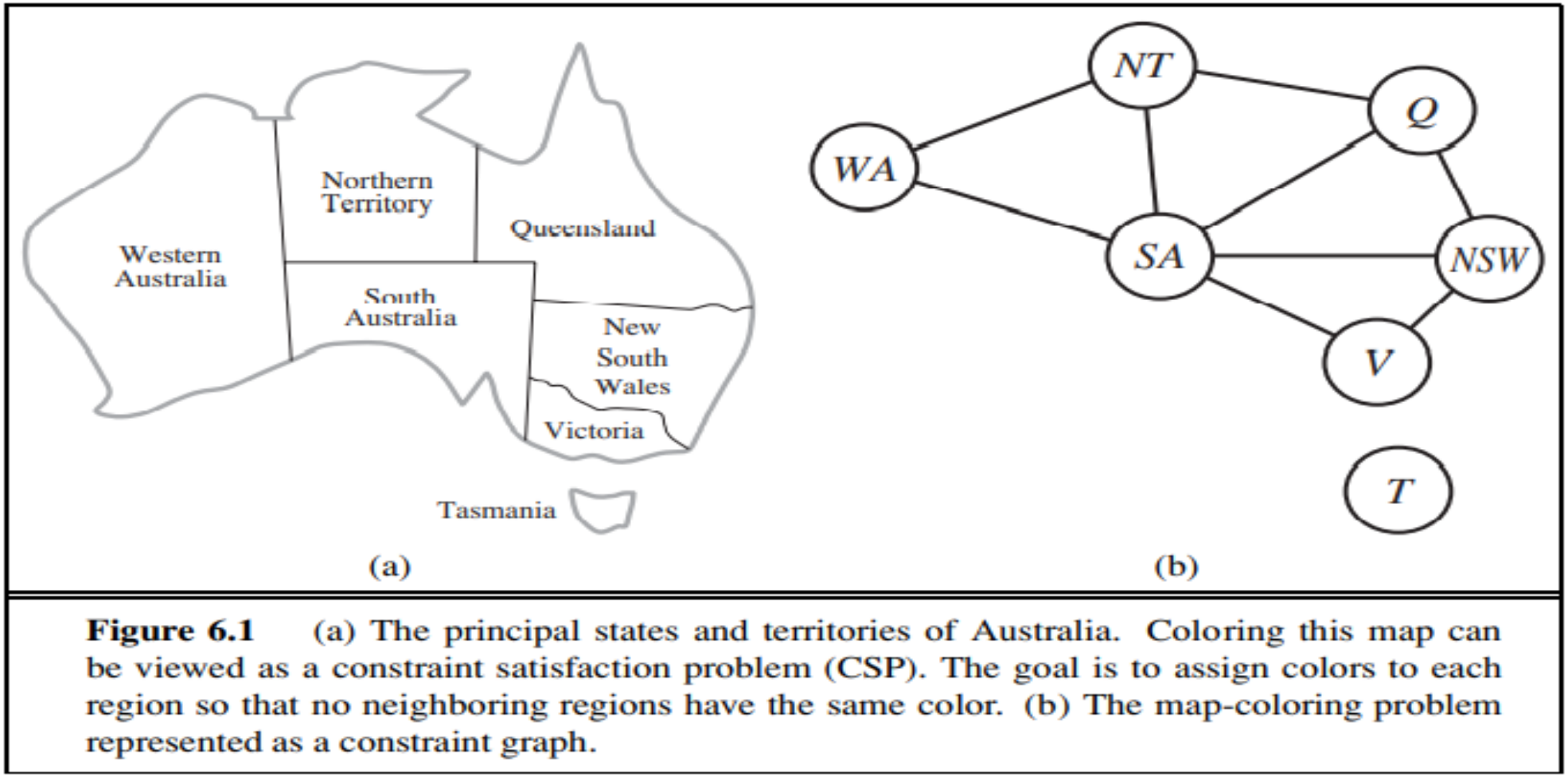
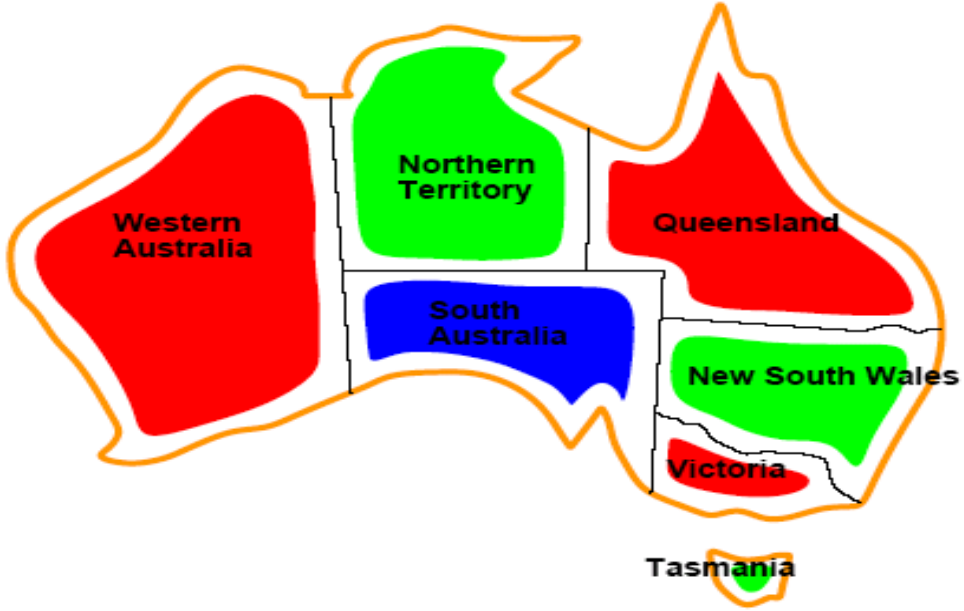


Figure 6.1 (a) The principal states and territories of Australia. Coloring this map can be viewed as a constraint satisfaction problem (CSP). The goal is to assign colors to each region so that no neighboring regions have the same color. (b) The map-coloring problem represented as a constraint graph.



- Solutions are assignments satisfying all constraints, e.g.

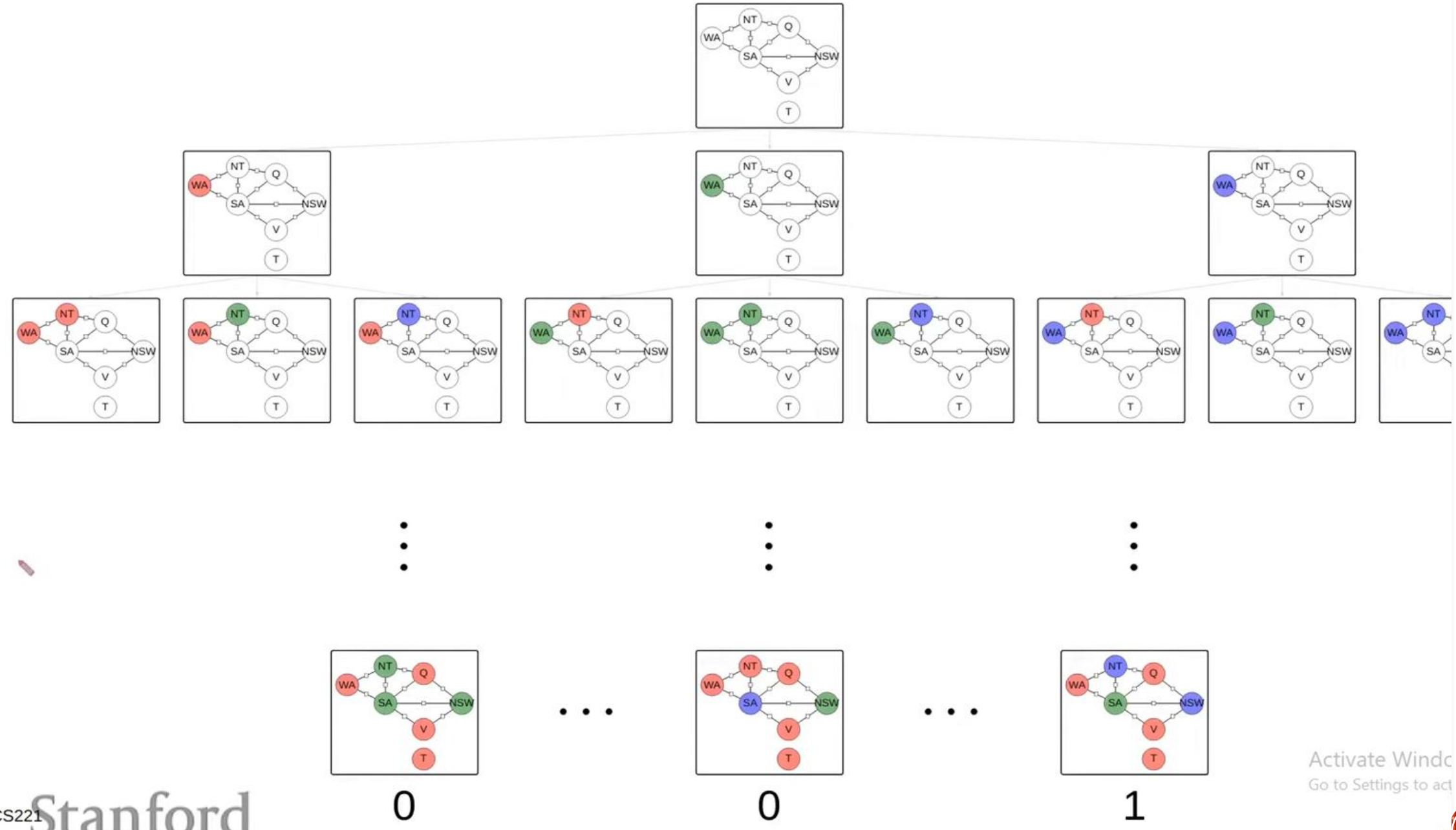
  {WA=red,NT=green,Q=red,NSW=green,V=red,SA=blue,T=green}

CONSTRAINT TYPES IN CSP

- **Unary Constraints:** It is the simplest type of constraints that restricts the value of a single variable.
  - Example: In the **map-coloring** problem
  - Variable: SA (South Australia)
  - Constraint: SA ≠ Green (South Australians don't want their region colored green)
  - So the domain of SA changes from {Red, Green, Blue} → {Red, Blue}.
- **Binary Constraints:** It is the constraint type which relates two variables.
  - Example, SA≠ NSW is a binary constraint.
- **Global Constraints:** It is the constraint type which involves an arbitrary number of variables.
  - **AllDifferent(X1, X2, ..., Xn):** All variables must take different values.
  - Used in **Sudoku** (each row/column/box must have different digits).

CONSTRAINT PROPAGATION

- **Using the constraints to reduce the number of legal values for a variable, which in turn can reduce the legal values for another variable, and so on**
- **Example- Map Colouring Problem (Australia)**
  - Variables: X={WA,NT,SA,Q,NSW,V,T}
  - Domain: {Red, Green, Blue}
  - Constraint: Neighbouring regions must have different colours.
  - If we assign WA=Red, then immediately:
    - NT and SA cannot be Red.
    - Their domains shrink from {R,G,B}→ {G,B}.
- This restriction **propagates** further.
- The idea behind constraint propagation is **local consistency.**
- *The process of enforcing local consistency in each part of the graph causes inconsistent values to be eliminated throughout the graph.*



- In local consistency, variables are treated as **nodes**, and each binary constraint is treated as an **arc** in the given problem.
- **There are following local consistencies**
- **Node Consistency:** A single variable is said to be node consistent if all the values in the variable's domain satisfy the unary constraints on the variables.
  Eg:-SA ≠ green
  - where South Australians dislike green, the variable SA starts with domain {red, green, blue}, and we can make it node consistent by eliminating green, leaving SA with the reduced domain {red, blue}.

- **Arc Consistency:** A variable is arc consistent if every value in its domain satisfies the binary constraints of the variables.
- **X_i is arc-consistent with respect to another variable X_j if for every value in the current domain D_i there is some value in the domain D_j that satisfies the binary constraint on the arc (X_i,X_j).**
- A network is arc-consistent if every variable is arc consistent with every other variable.
- For example, consider the constraint $Y = X^2$ where the domain of both X and Y is the set of digits. We can write this constraint explicitly as (X, Y), {(0, 0),(1, 1),(2, 4),(3, 9))} .
- To make X arc-consistent with respect to Y , we reduce X's domain to {0, 1, 2, 3}. If we also make Y arc-consistent with respect to X, then Y 's domain becomes {0, 1, 4, 9} and the whole CSP is arc-consistent.

AC-3 ALGORITHM FOR ARC CONSISTENCY

There is a queue of arcs to make every variable arc-consistent

1. Initially, the queue contains all the arcs in the CSP
2. AC-3 then pops off an arbitrary arc (Xi, Xj) from the queue and makes Xi arc-consistent with respect to Xj
   Call REVISE(X_i, X_j(remove any value $x \in D(X_i)$ that has no supporting $y \in D(X_j)$ satisfying the constraint between X_i and X_j.
3. If Di is unchanged, the algorithm just moves on to the next arc
4. B)Else if this revises Di (makes the domain smaller),
   a. then we add to the queue all arcs (Xk, Xi) where Xk is a neighbor of Xi
      - because the change in Di might enable further reductions in the domains of Dk
      - even if we have previously considered Xk
5. Else if Di is revised down to nothing, ***If $D(X_i)$ becomes empty → return failure (no solution).***
   a. then we know the whole CSP has no consistent solution, and AC-3 can immediately return failure

AC3 ALGORITHM

6. Otherwise, we keep checking, trying to remove values from the domains of variables until no more arcs are in the queue
7. At that point, we are left with a CSP that is equivalent to the original CSP
   - Then they both have the same solutions but
   - the arc-consistent CSP will in most cases be faster to search because its variables have smaller domains.

- Path consistency tightens the binary constraints by using implicit constraints that are inferred by looking at triples of variables.
- **Path Consistency:** When the evaluation of a set of two variable with respect to a third variable can be extended over another variable, satisfying all the binary constraints. It is similar to arc consistency.

  - **A two-variable set {Xi, Xj} is path-consistent with respect to a third variable Xm if,**
    - **For every assignment {Xi = a, Xj = b} consistent with the constraints on {Xi, Xj},**
    - **There is an assignment to Xm that satisfies the constraints on {Xi, Xm} and {Xm, Xj}.**

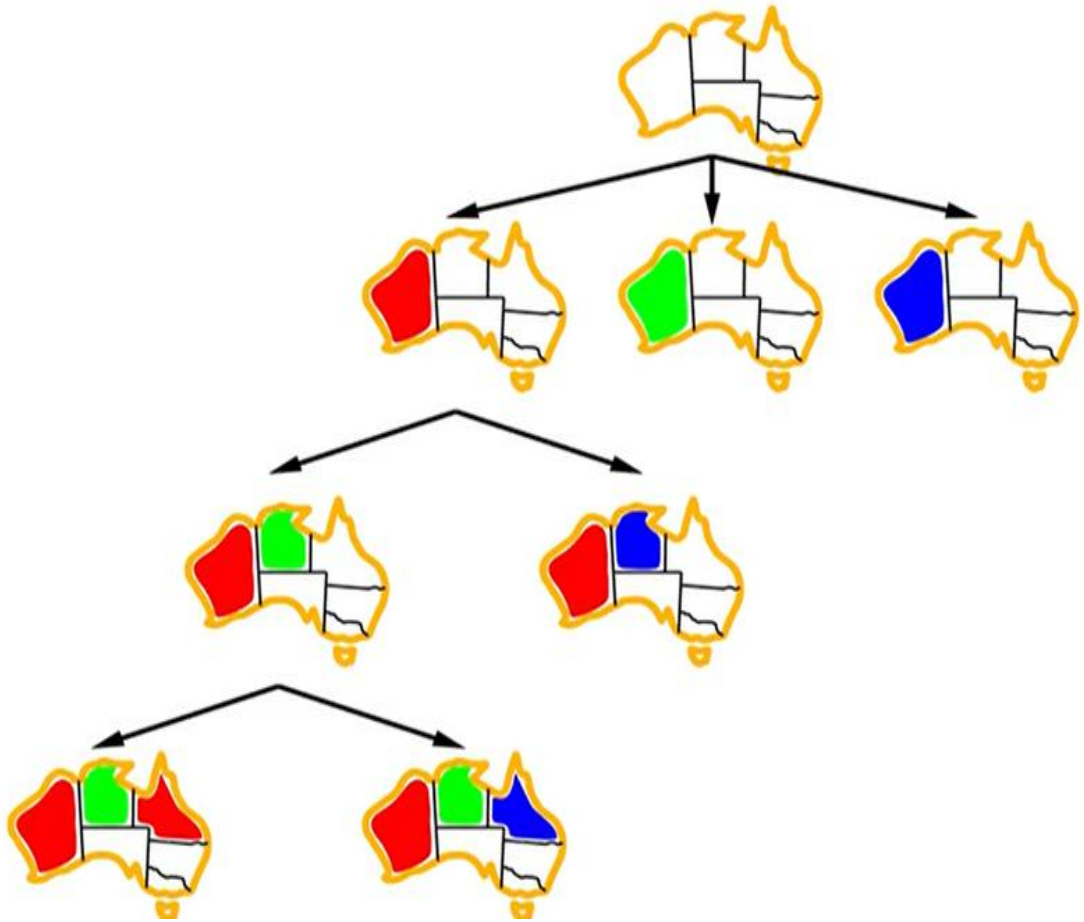
  - This is called path consistency because one can think of it as looking at a path from Xi to Xj with Xm in the middle.

- **k-consistency:** This type of consistency is used to define the notion of stronger forms of propagation. Here, we examine the k-consistency of the variables.
- Example :-
- First, we choose a consistent value for X1.
- We are then guaranteed to be able to choose a value for X2 because the graph is 2-consistent,
- for X3 because it is 3-consistent, and so on.
- For each variable Xi, we need only search through the d values in the domain to find a value consistent with X1,...,Xi−1.
- We are guaranteed to find a solution in time $O(n^2d)$.

BACKTRACKING SEARCH

- It is a depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign
- It repeatedly chooses an unassigned variable, and then tries all values in the domain of that variable in turn, trying to find a solution.
- If an inconsistency is detected, then BACKTRACK returns failure, causing the previous call to try another value.

Backtracking example



BACKTRACKING SEARCH

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences ← INFERENCE(csp, var, value)
      if inferences ≠ failure then
        add inferences to assignment
        result ← BACKTRACK(assignment, csp)
        if result ≠ failure then
          return result
    remove {var = value} and inferences from assignment
  return failure
```

BACKTRACKING SEARCH

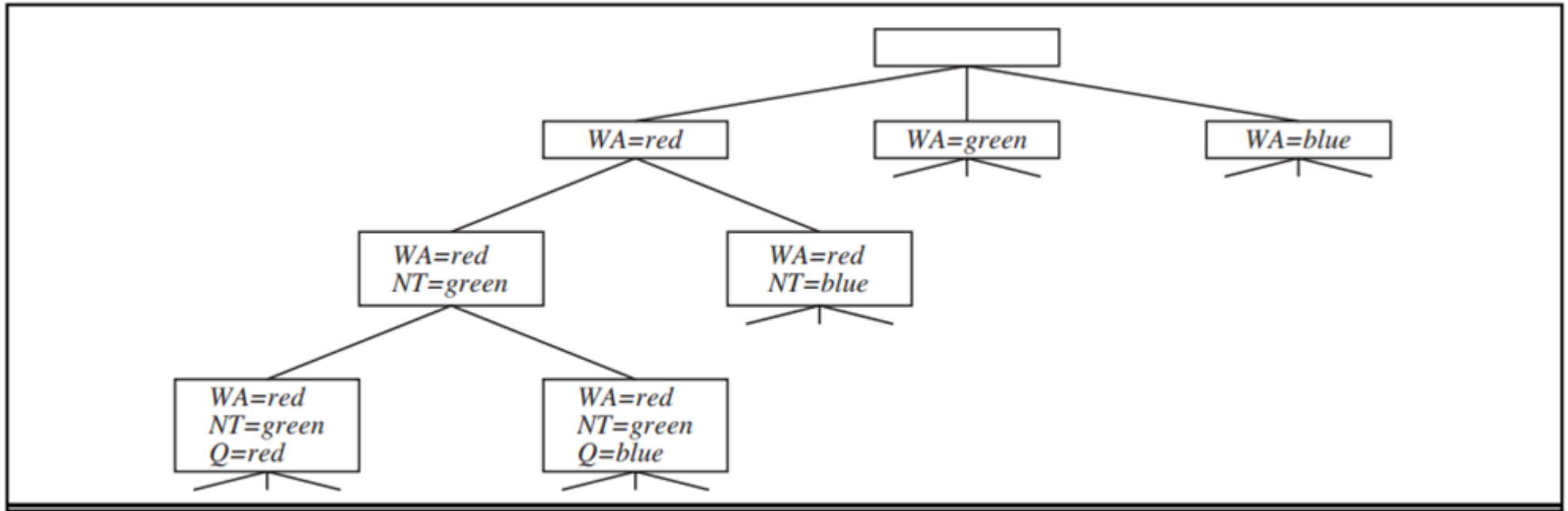


Figure 6.6 Part of the search tree for the map-coloring problem in Figure 6.1.

BACKTRACKING-SEARCH keeps only a single representation of a state and alters that representation rather than creating new ones

Improving backtracking efficiency

General-purpose methods can give huge gains in speed:

1. Which variable should be assigned next?
2. In what order should its values be tried?
3. Can we detect inevitable failure early?
4. Can we take advantage of problem structure?

BACKTRACKING SEARCH

To solve CSPs efficiently without domain-specific knowledge, address following questions:

1) function SELECT-UNASSIGNED-VARIABLE: which variable should be assigned next?

2)function ORDER-DOMAIN-VALUES: in what order should its values be tried?

3)function INFERENCE: what inferences should be performed at each step in the search?

4) When the search arrives at an assignment that violates a constraint, can the search avoid repeating this failure?

BACKTRACKING SEARCH

Variable and value ordering

var ← SELECT-UNASSIGNED-VARIABLE(csp)

The simplest strategy for SELECT-UNASSIGNED-VARIABLE is to choose the next unassigned variable in order, {X1, X2,...}.

This static variable ordering seldom results in the most efficient search.

Minimum-remaining-values (MRV) heuristic:

The idea of choosing the variable with the fewest “legal” value.

A.k.a. “**most constrained variable**” or “fail-first” heuristic,

it picks a variable that is most likely to cause a failure soon thereby pruning the search tree.

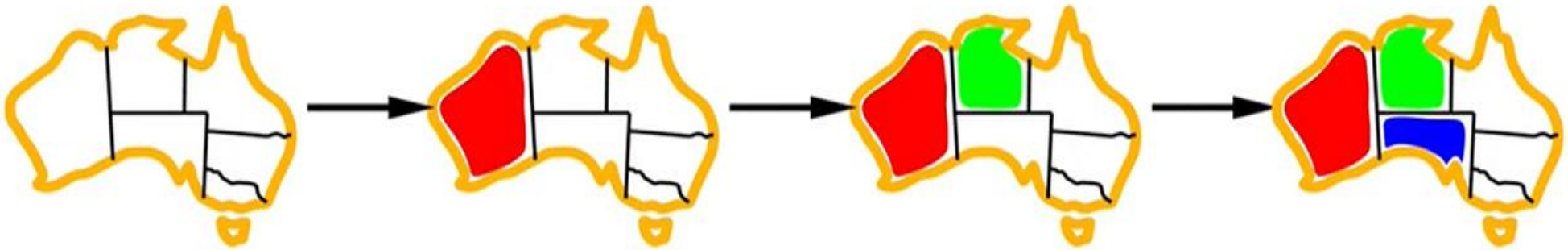
If some variable X has no legal values left, the MRV heuristic will select X and failure will be detected immediately avoiding pointless searches through other variables.

E.g. After the assignment for WA=red and NT=green, there is only one possible value for SA, so it makes sense to assign SA=blue next rather than assigning Q.

Minimum remaining values

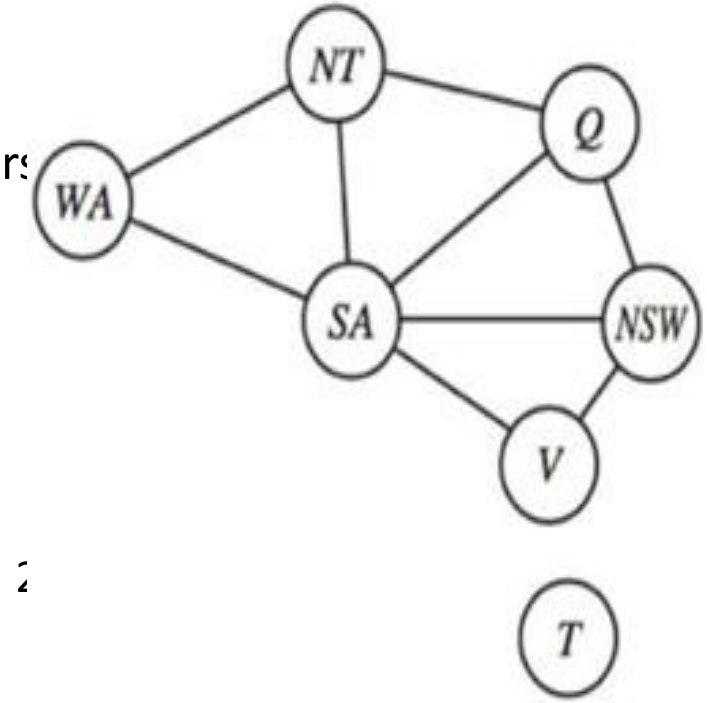
Minimum remaining values (MRV):

choose the variable with the fewest legal values



DEGREE HEURISTIC

- The MRV heuristic doesn't help at all in choosing the first region to colour in Australia, because initially every region has three legal colours
- Degree heuristic attempts to reduce the branching factor on future choices by selecting the variable that is involved in the largest number of constraints on other unassigned variables.
- EXAMPLE
  - SA is the variable with highest degree 5; the other variables have degree 2 or 3; T has degree 0.
  - Once SA is chosen, applying the degree heuristic solves the problem without any false steps you can choose any consistent colour at each choice point and still arrive at a solution with no backtracking

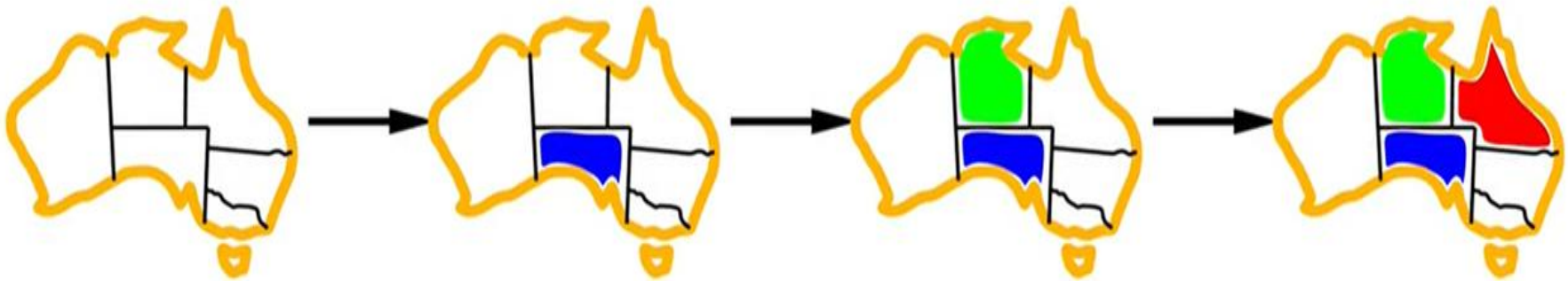


Degree heuristic

Tie-breaker among MRV variables

Degree heuristic:

choose the variable with the most constraints on remaining variables



LEAST CONSTRAINING VALUE

we have generated the partial assignment with WA = red and NT = green and that our next choice is for Q.

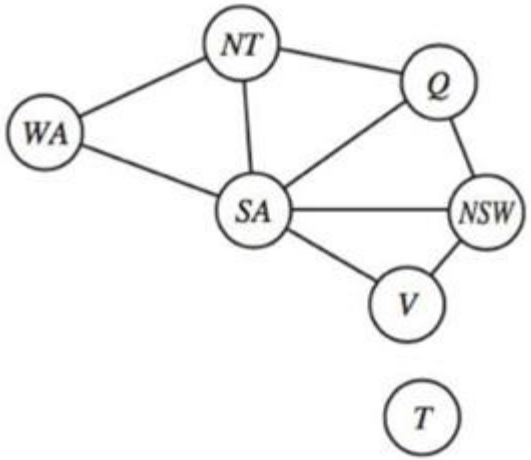
Blue would be a bad choice because it eliminates the last legal value left for Q's neighbor, SA.

The least-constraining-value heuristic therefore prefers red to blue.

the heuristic is trying to leave the maximum flexibility for subsequent variable assignments.

Of course, if we are trying to find all the solutions to a problem, not just the first one, then the ordering does not matter because we have to consider every value anyway.

The same holds if there are no solutions to the problem



Least constraining value

Given a variable, choose the least constraining value:
the one that rules out the fewest values in the remaining variables

Allows 1 value for SA

Allows 0 values for SA

Interleaving search and inference

But inference can be even more powerful in the course of a search: every time we make a choice of a value for a variable, we have a brand-new opportunity to infer new domain reductions on the neighboring variables.

FORWARD CHECKING

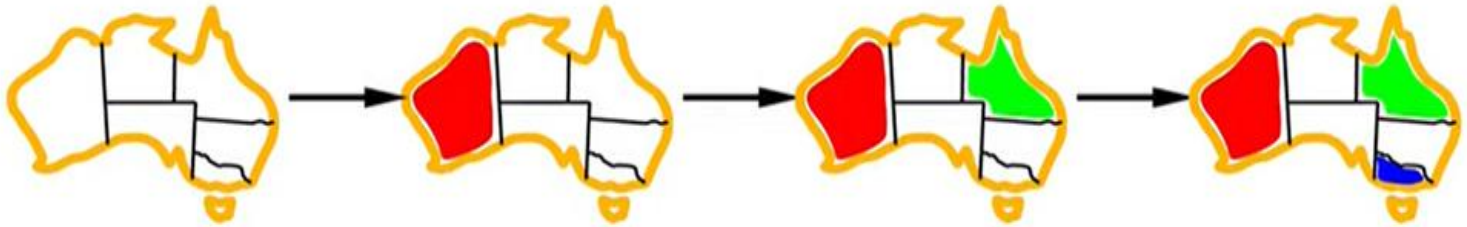
One of the simplest forms of inference.

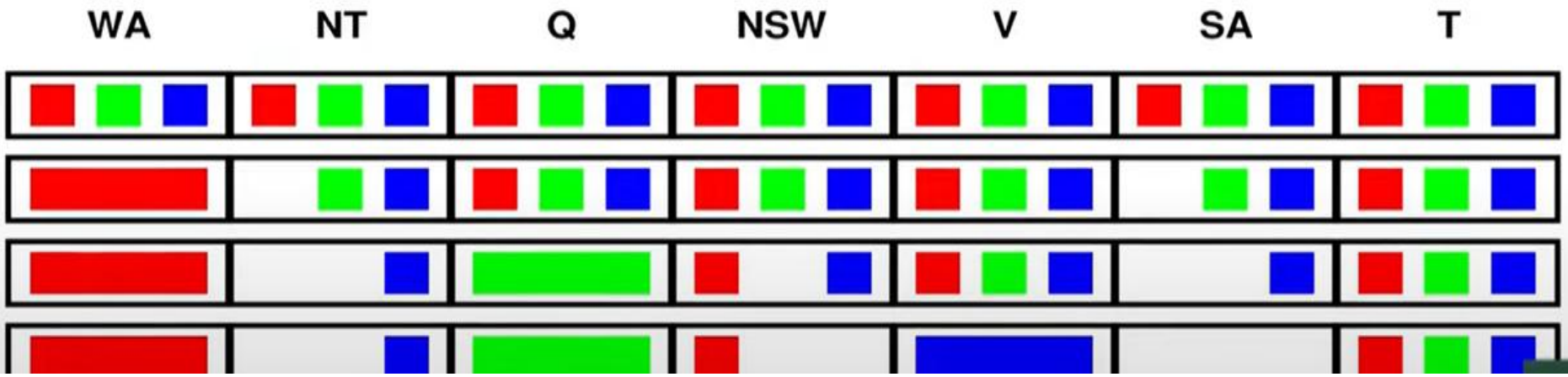
Whenever a variable X is assigned, the forward-checking process establishes arc consistency for it: for each unassigned variable Y that is connected to X by a constraint, delete from Y's domain any value that is inconsistent with the value chosen for X.

There is no reason to do forward checking if we have already done arc consistency as a preprocessing step.

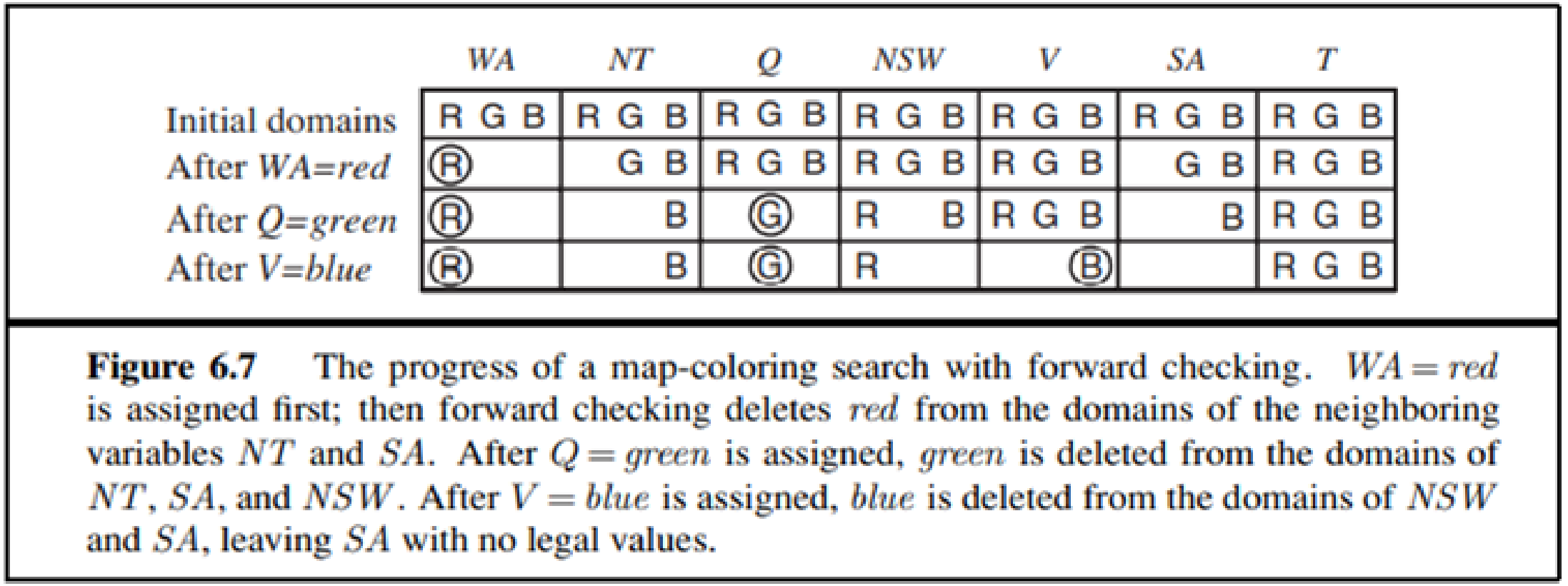
Forward checking

Idea: Keep track of remaining legal values for unassigned variables
Terminate search when any variable has no legal values





FORWARD CHECKING



	WA	*NT*	*Q*	*NSW*	*V*	*SA*	*T*
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After *WA*=*red*	(R)	G B	R G B	R G B	R G B	G B	R G B
After *Q*=*green*	(R)	B	(G)	R B	R G B	B	R G B
After *V*=*blue*	(R)	B	(G)	R	(B)		R G B

Figure 6.7 The progress of a map-coloring search with forward checking. $WA = red$ is assigned first; then forward checking deletes *red* from the domains of the neighboring variables NT and SA. After $Q = green$ is assigned, *green* is deleted from the domains of NT, SA, and NSW. After $V = blue$ is assigned, *blue* is deleted from the domains of NSW and SA, leaving SA with no legal values.

FORWARD CHECKING

There are two important points to notice about this example.

First, notice that after WA = red and Q = green are assigned, the domains of NT and SA are reduced to a single value; we have eliminated branching on these variables altogether by propagating information from WA and Q.

A second point to notice is that after V = blue, the domain of SA is empty.

Hence, forward checking has detected that the partial assignment {WA = red, Q = green, V = blue} is inconsistent with the constraints of the problem, and the algorithm will therefore backtrack immediately

FORWARD CHECKING

Advantage

- For many problems the search will be more effective if we combine the MRV heuristic with forward checking.

Disadvantage

- Forward checking only makes the current variable arc- consistent, but doesn't look ahead and make all the other variables arc-consistent.

MAC (Maintaining Arc Consistency) algorithm

More powerful than forward checking, detect this inconsistency.

After a variable X_i is assigned a value, the INFERENCE procedure calls AC-3, but instead of a queue of all arcs in the CSP, we start with only the arcs(X_j, X_i) for all X_j that are unassigned variables that are neighbors of X_i.

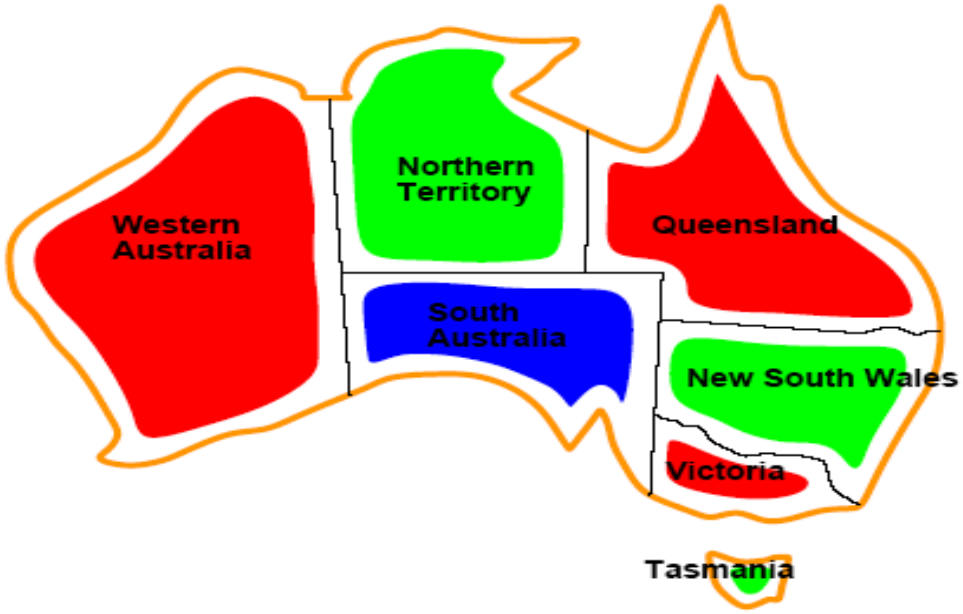
From there, AC-3 does constraint propagation in the usual way, and if any variable has its domain reduced to the empty set, the call to AC-3 fails and we know to backtrack immediately.

Constraint propagation

Forward checking propagates information from assigned to unassigned vari ables, but doesn't provide early detection for all failures:

NT and SA cannot both be blue!

Constraint propagation repeatedly enforces constraints locally



Solutions are assignments satisfying all constraints, e.g.

{WA=red,NT=green,Q=red,NSW=green,V=red,SA=blue,T=green}

SUDOKU EXAMPLE

- **Sudoku Playing:** The gameplay where the constraint is that no number from 0-9 can be repeated in the same row or column.

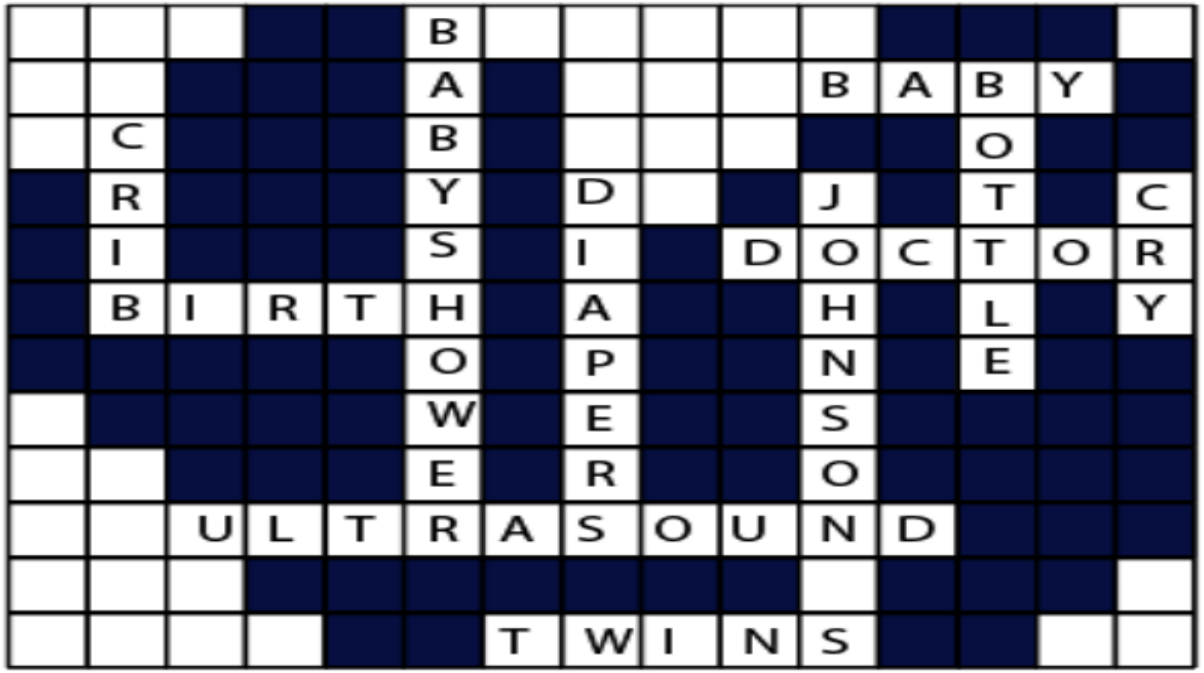
4							5	9
2	6		5				3	
				9	2			
		2		6			1	
		3	8	1	9	7		
	7			3		5		
			3	4				
	3				6		2	7
5	9							6

Puzzle

4	1	7	6	8	3	2	5	9
2	6	9	5	7	1	8	3	4
3	8	5	4	9	2	6	7	1
8	4	2	7	6	5	9	1	3
6	5	3	8	1	9	7	4	2
9	7	1	2	3	4	5	6	8
7	2	6	3	4	8	1	9	5
1	3	8	9	5	6	4	2	7
5	9	4	1	2	7	3	8	6

Solution

- **n-queen problem:** In n-queen problem, the constraint is that no queen should be placed either diagonally, in the same row or column.
- **Crossword:** In crossword problem, the constraint is that there should be the correct formation of the words, and it should be meaningful.



CRYPTARITHMETIC PROBLEM

- Cryptarithmetic Problem is a type of ***constraint satisfaction problem***
- The game is about digits and its unique replacement either with alphabets or other symbols.
- In cryptarithmetic problem, the digits (0-9) get substituted by some possible alphabets or symbols.
- The task in cryptarithmetic problem is to substitute each digit with an alphabet to get the result arithmetically correct.

- A **cryptarithmetic problem** is a type of **constraint satisfaction problem (CSP)** where the goal is to assign digits (0–9) to letters in an arithmetic problem so that the arithmetic holds true.
- Each letter represents a unique digit.
- No two letters can have the same digit.
- Numbers cannot start with zero.

EXAMPLE

- **Question 1 : SEND + MORE = MONEY**
- Solution : 9567+1085=10652
- So: **S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2**

- **Question 2 : TWO + TWO = FOUR**
- 734+734 =1468
- So: **T=7, W=3, O=4, F=1, U=6, R=8**

- **Question 3 : CROSS + ROADS = DANGER**
- 96233+62513=158746
- So: **C=9, R=6, O=2, S=3, A=5, D=1, N=8, G=7, E=4**

STRUCTURE OF CSP PROBLEMS

- Ways in which the structure of the problem, as represented by the constraint graph, can be used to find solutions quickly.
- CSPs can be solved more efficiently by analyzing the **constraint graph**.
- Goal: reduce exponential search into smaller, tractable subproblems.

DECOMPOSING INTO INDEPENDENT SUBPROBLEMS

- Sometimes, parts of your problem are completely separate.
- Think of coloring a map: if an island like Tasmania isn't connected to the mainland, then coloring Tasmania has absolutely no effect on coloring the mainland.
- This means you can solve these parts (subproblems) completely independently.
- If you have n variables in total and they break into n/c independent subproblems, the time it takes to solve the whole thing can be drastically reduced.
- You can figure out if parts are independent by looking at the constraint graph – if they are not connected, they are independent.

TREE-STRUCTURED CSPS

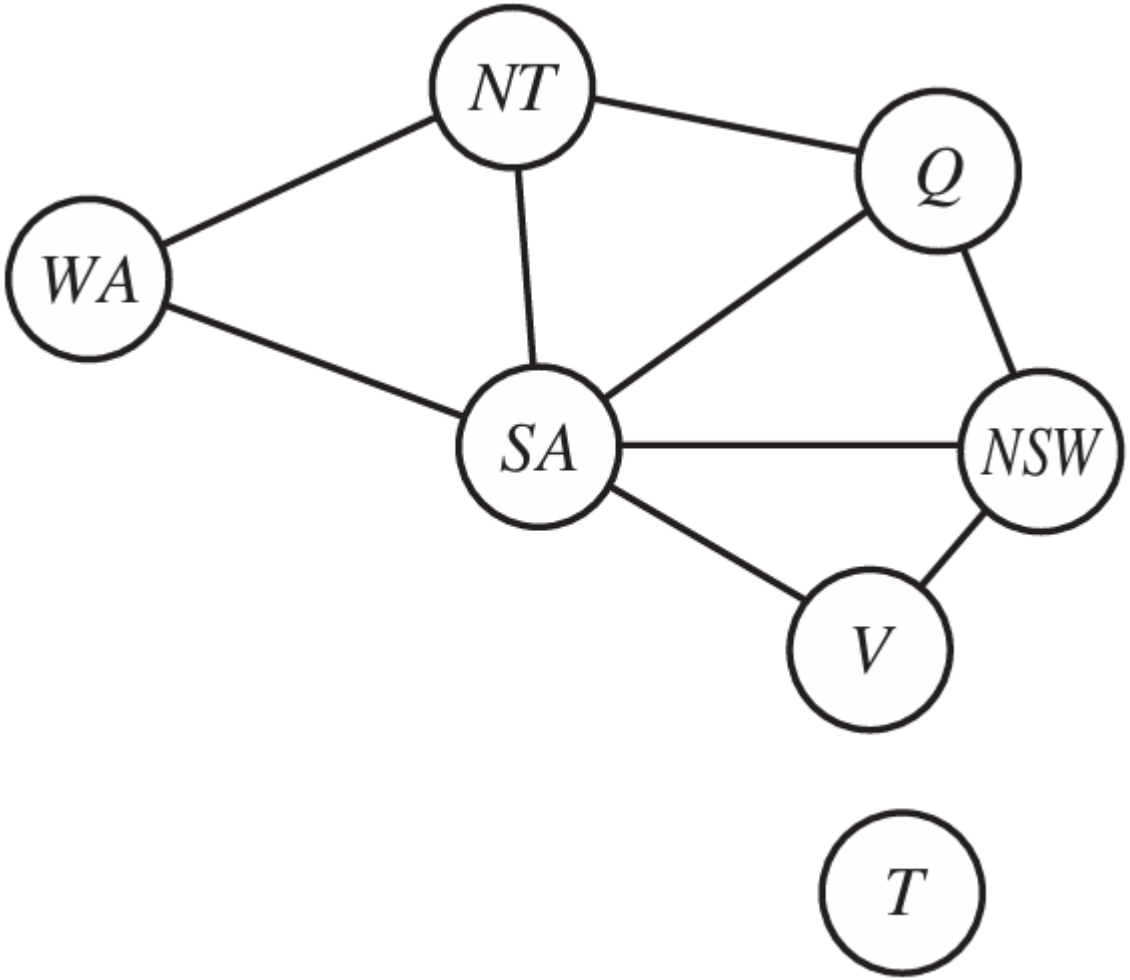
- Many real-world problems aren't completely independent, but some have a structure like a tree.
- **In a tree, any two variables are connected by only one path.**
- Problems with this "tree-like" structure are easy to solve very quickly, in a time that grows linearly with the number of variables.
- The key is something called Directed Arc Consistency (DAC). Imagine you order the variables in a specific way, like a family tree where children come after their parents. DAC ensures that for any value you pick for a "parent" variable, there will always be a valid value for its "child" variable.
- Once you've made the problem DAC (which takes $O(nd^2)$ time), you can simply go through the variables in order and pick any valid value.
- You won't get stuck and have to backtrack, making the process very fast and linear.
- **This specific ordering is called a topological sort.**

TURNING COMPLEX PROBLEMS INTO TREE-LIKE PROBLEMS

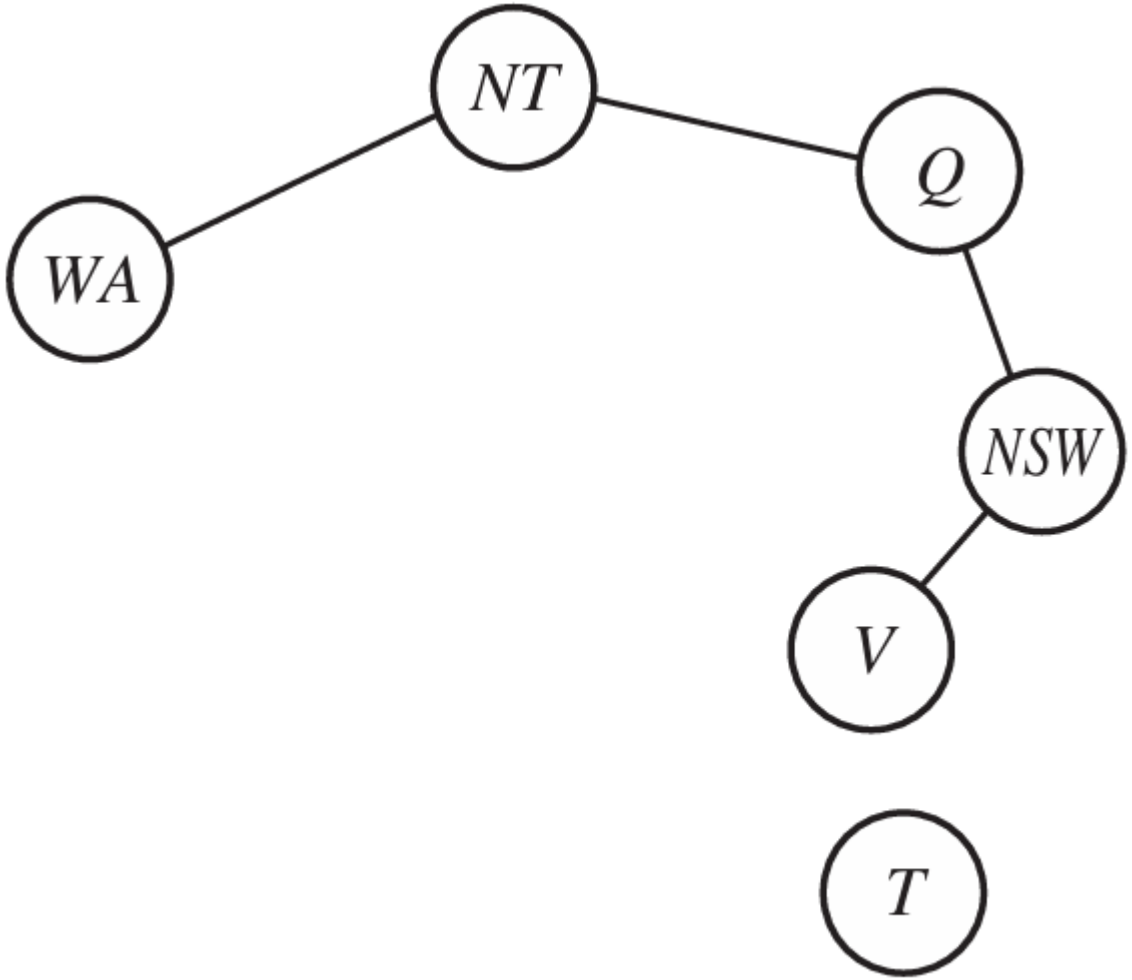
- Most problems aren't simple trees, but we can often make them behave like trees using two main strategies:
- **a. Cutset Conditioning (Removing Variables):**
  - The idea is to find a small group of "critical" variables (called a cycle cutset) whose removal would turn the rest of the problem into a tree
  - For instance, if you removed South Australia from the Australia map, the remaining connections would form a tree.
  - You then try out all possible assignments for these critical variables in the cycle cutset.
  - For each assignment, you solve the remaining (now tree-structured) problem using the fast tree algorithm.
  - If the cycle cutset is small, this approach provides huge savings compared to trying all possibilities directly. This technique is also known as **cutset conditioning**

CUTSET CONDITIONING

1. Choose a subset S of the CSP's variables such that the constraint graph becomes a tree after removal of S. S is called a **cycle cutset**.
2. For each possible assignment to the variables in S that satisfies all constraints on S,
   (a) remove from the domains of the remaining variables any values that are inconsistent with the assignment for S, and
   (b) If the remaining CSP has a solution, return it together with the assignment for S.



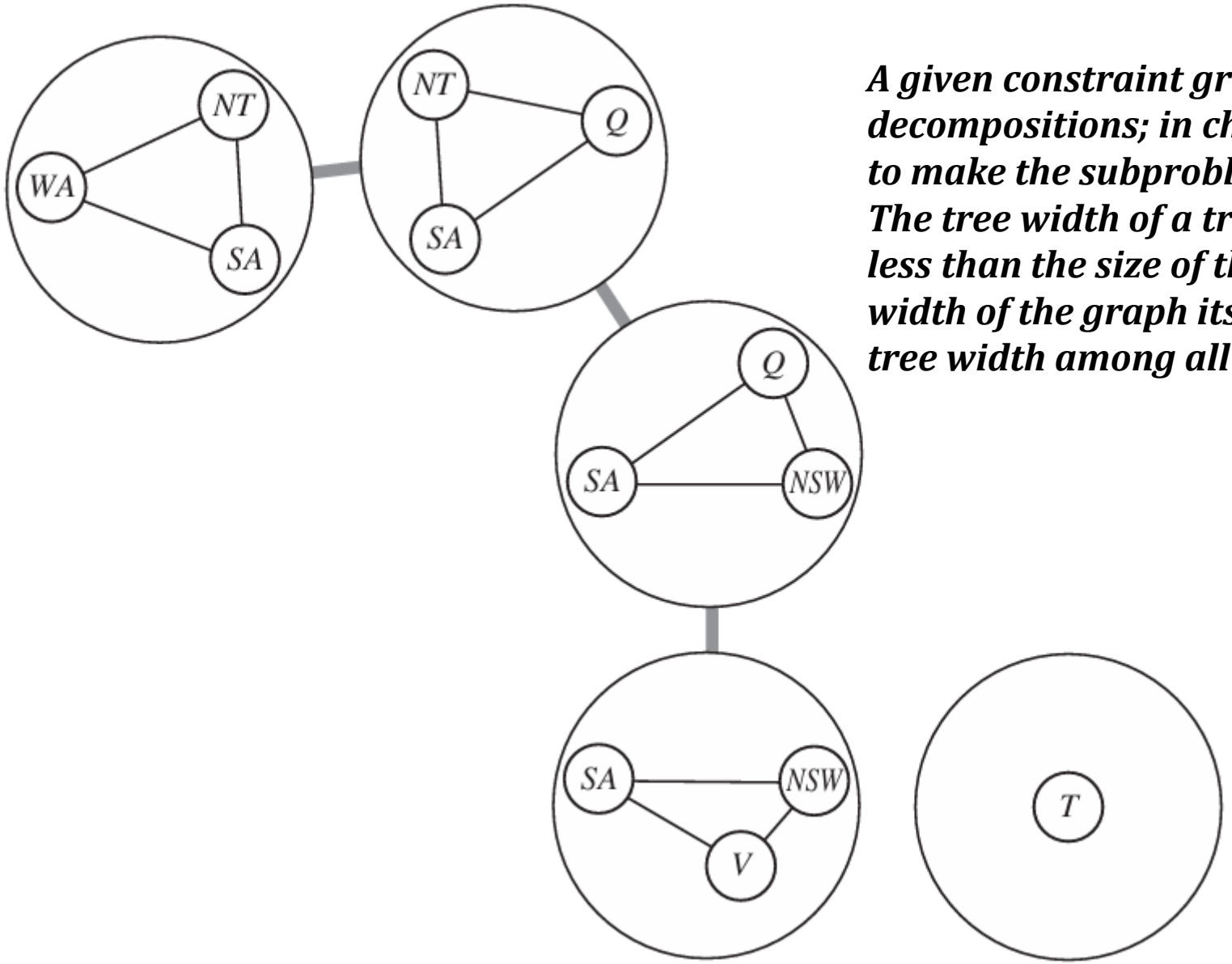
(a)



(b)

a) The original constraint graph (b) The constraint graph after the removal of SA

- **b. Tree Decomposition (Collapsing Variables):**
- This method breaks the original problem into several smaller, connected subproblems. These subproblems themselves are then arranged in a tree structure.
- There are specific rules for how to create these subproblems:
  - **Coverage**: Every original variable must be in at least one subproblem.
  - **Constraint preservation**: If two variables are linked by a constraint, they must appear together in at least one subproblem.
  - **Running intersection property**: If a variable appears in multiple subproblems, it must appear in all subproblems along the path connecting them in the decomposition tree.
  - Example : In the Australia map-coloring problem, South Australia (SA) appears in multiple subproblems, and it must remain consistent across them.
  - Each subproblem is solved independently. Then, these solutions are combined using the efficient tree algorithm, where each subproblem is treated like a "mega-variable".
  - The tree width of a graph tells you how "tree-like" it is.
  - Problems with a small tree width can be solved in polynomial time, meaning they are much more efficient.

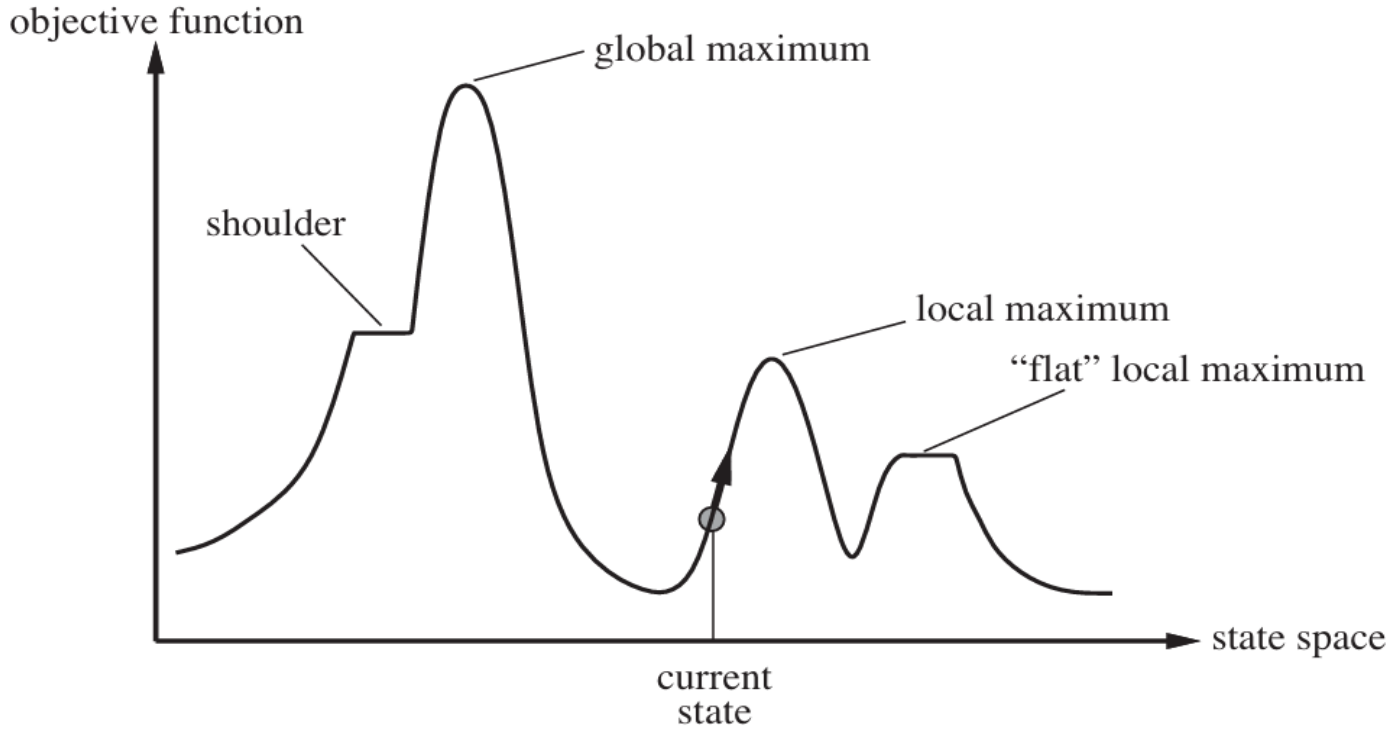


A given constraint graph admits many tree decompositions; in choosing a decomposition, the aim is to make the subproblems as small as possible.
The tree width of a tree decomposition of a graph is one less than the size of the largest subproblem; the tree width of the graph itself is defined to be the minimum tree width among all its tree decompositions.

A tree decomposition of the constraint graph

HILL CLIMBING ALGORITHM

- Hill climbing algorithm is a **local search algorithm** which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem.
- It terminates when it reaches a peak value where no neighbor has a higher value.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
  - Hill Climbing is mostly used when a good heuristic is available.
  - In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.
- The idea behind hill climbing is as follows.
- 1. Pick a random point in the search space.
- 2. Consider all the neighbors of the current state.
- 3. Choose the neighbor with the best quality and move to that state.
- 4. Repeat 2 to 4 until all the neighboring states are of lower quality.
- 5. Return the current state as the solution state.



function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current ← MAKE-NODE(*problem*.INITIAL-STATE)
loop do
neighbor ← a highest-valued successor of *current*
if neighbor.VALUE ≤ current.VALUE **then return** *current*.STATE
current ← *neighbor*

HILL CLIMBING ALGORITHM

- **Different regions in the state space landscape:**
- **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.
- **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.
- **Plateau (Flat local Maximum):** A flat region where many neighboring states have the same value. The algorithm may wander randomly or stop because it sees no "uphill" improvement.
- **Current state:** It is a state in a landscape diagram where an agent is currently present.
- **Ridge:** A narrow path leading upward that is difficult to follow .Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate.

VARIANTS OF HILL CLIMBING

- Steepest-Ascent Hill Climbing
  - Always choose the best among all neighbors.
- Simple Hill Climbing
  - Choose the first better neighbor you find.
- Stochastic Hill Climbing
  - Pick a random better neighbor (adds randomness to escape traps).